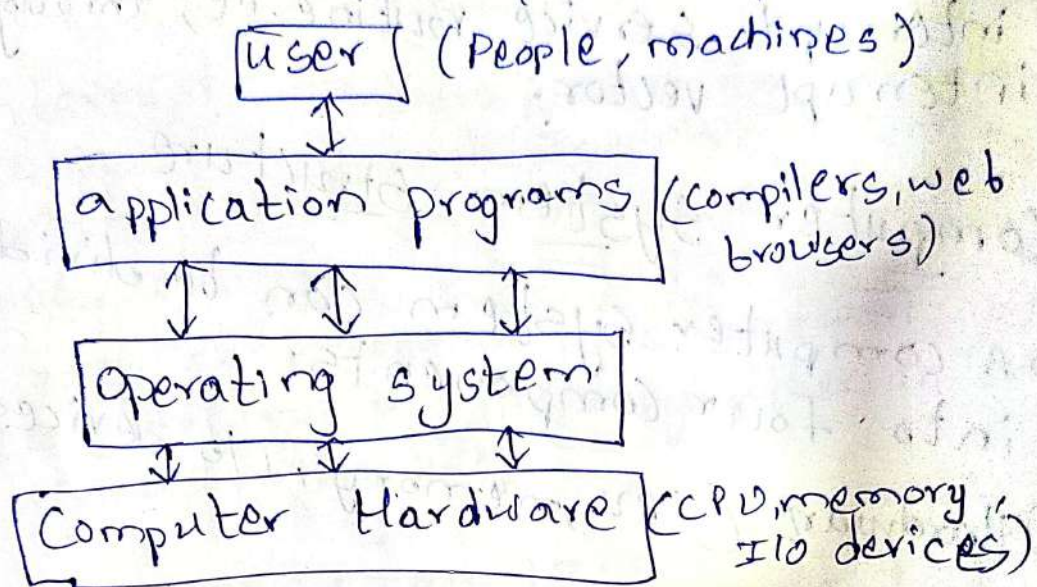


o.g:- A program that acts as an intermediary b/w a user of a computer & the computer hardware.



- Users want convenience, ease of use, & good performance.
- Shared computer such as main frame or mini computer.
- O.S is a resource allocator and control program, making efficient use of HW & managing execution of user programs.

Examples:-

- Dos, Windows, UNIX, Linux, Mac, Android, & iOS. (Graphical user interface)
- Oracle, Solaris, FreeBSD, Microsoft Xenix, IBM AIX, HP-UX.
- CentOS, Debian, Fedora, Kali Linux, Ubuntu, openSUSE. (Unix related).
- An O.S. is interrupt driven.
- Interrupt architecture must save the address of the interrupted instruction.

interrupt vector.

Computer System Structure

⇒ A computer system can be divided into four components:

- 1) Hardware (CPU, memory, I/O devices)
- 2) Operating System
(Controls & coordinates the use of hardware among various applications & users).
- 3) Application programs
(Word processors, compilers, web browsers, database systems, video games).
- 4) Users
(People, machines, other computers).

Interrupt

The device controller informs the CPU that it has finished its operation, causing an interrupt.

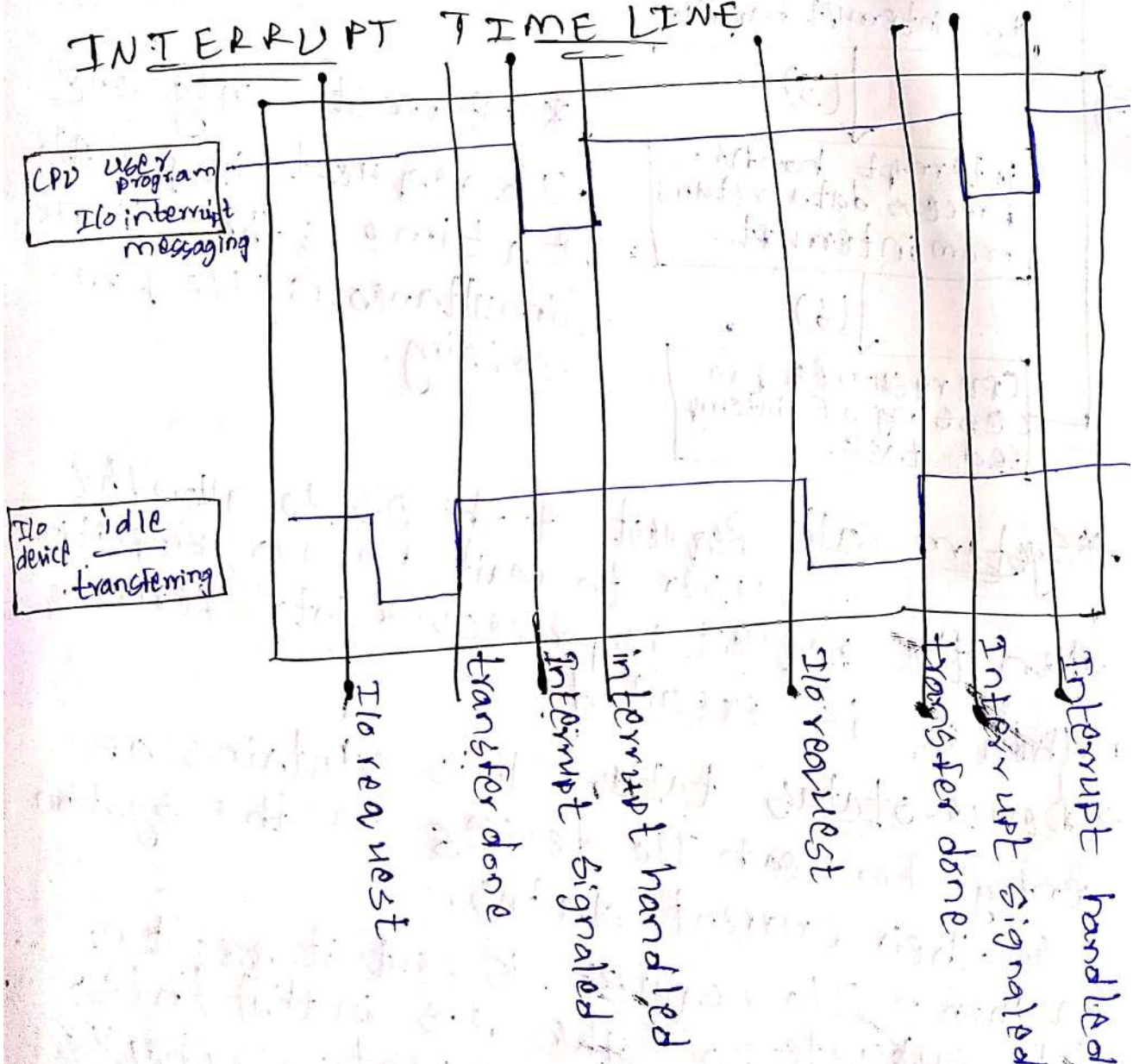
* The one program running at all times on the computer is the kernel (part of O.S). Where remaining programs are a "system programs" not part of the kernel (or) application programs (All programs are not associated with O.S).

* I/O device & device controllers and the CPU can execute concurrently.

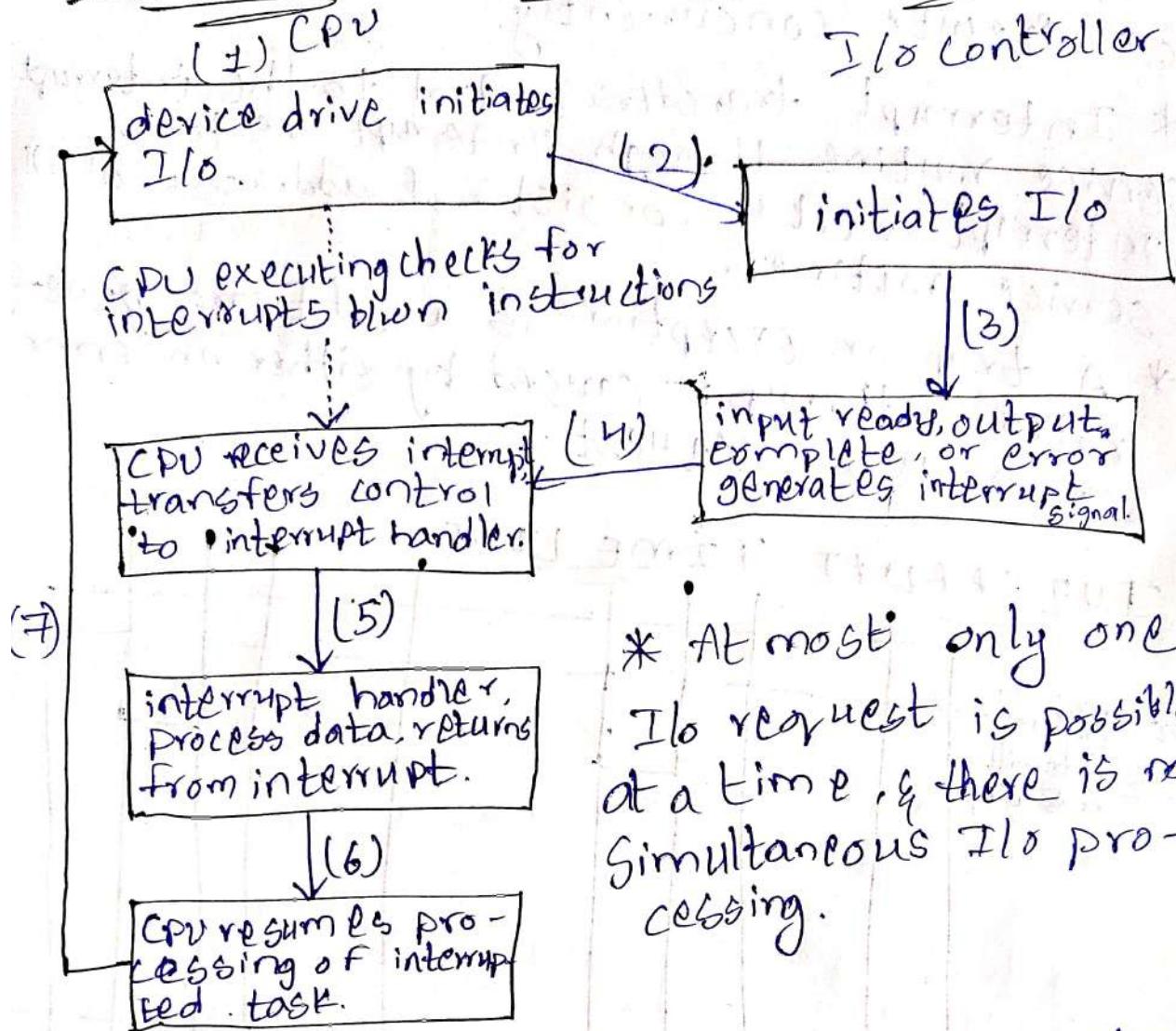
* Interrupt transfers control to the interrupt service routine through "Interrupt vector".
→ Interrupt vector consists of addresses of all service routines.

* A trap or exception is a software-generated interrupt caused by either an error or a user request.

INTERRUPT TIME LINE



INTERRUPT - DRIVE I/O CYCLE



* System call:- Request to the OS to allow the user to wait for I/O completion. then the request is processed into kernel & then it is executed.

* Device-status table:- This contains an entry for each I/O device in the system & their current status.

* When a I/O request is sent it goes to I/O subsystem of the OS in that this device status table is present. OS checks device entry in the Dst, If FREE → assigns that particular device & set its status = BUSY. If ~~status~~ BUSY then the process is kept in queue.

Storage Structure

26/07/25

→ Main memory

- * Random access memory in the form of (DRAM)
- * Typically volatile

→ Secondary storage

- * Extension of main memory
- * non-volatile
- * ~~Hard~~ Disk surface is logically divided into tracks, which are subdivided into sectors.

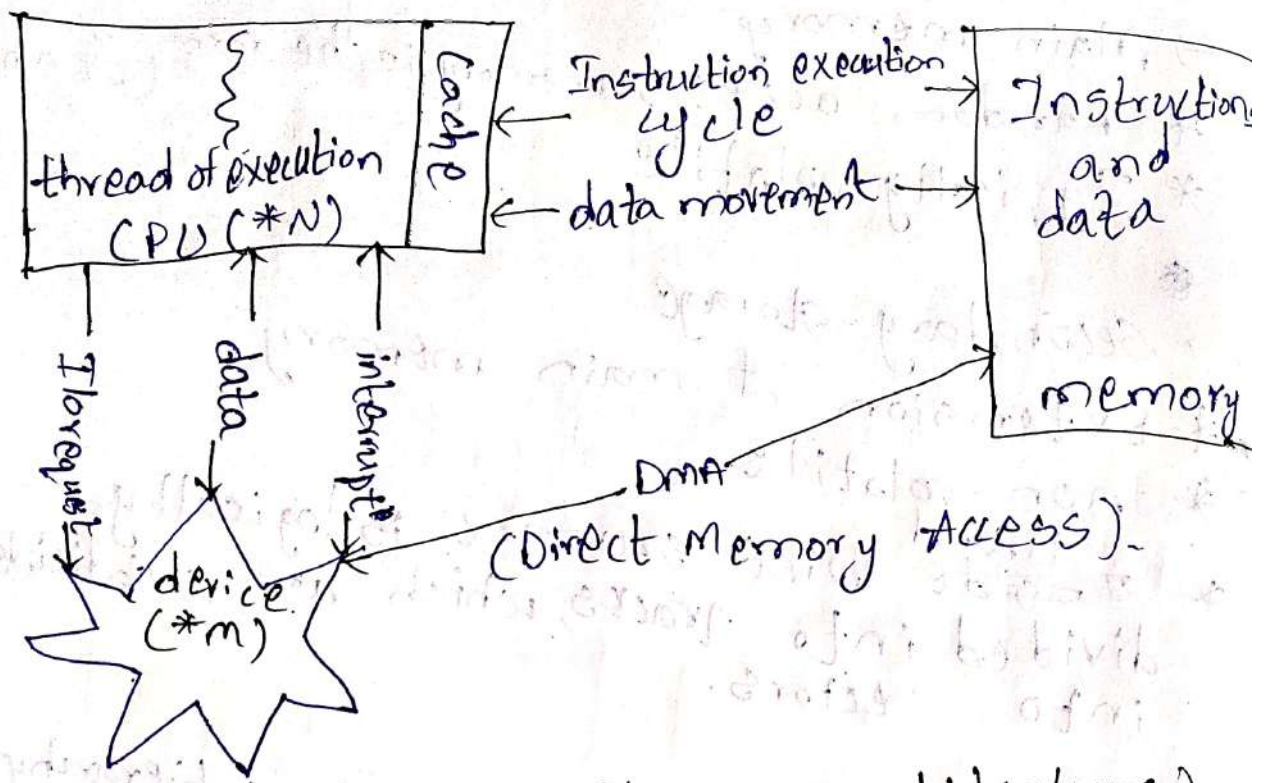
→ speed
cost
Volatility } Storage systems hierarchy.

→ Device driver for each device controller to manage I/O

- * Provides a uniform interface b/w the controller & the kernel.



How a modern computer works



(A Von Neumann architecture).

DMA structure

- Used for high speed I/O devices able to transmit information at close to memory speeds.
- Device controller transfers blocks of data from buffer storage to main memory.

O/S operations

- Bootstrap program (simple code to initialize the system, load the kernel).
- kernel loads.
- Starts system daemons (services provided outside of the kernel).
- kernel interrupt driven (hardware & software).

Multi programming jobs

→ This organizes (code & data) so the CPU always has one to execute.

Multi tasking :- A logical extension of Batch systems - the CPU switches jobs so frequently that users can interact with each job while it is running, creating interactive computing.

→ Response time

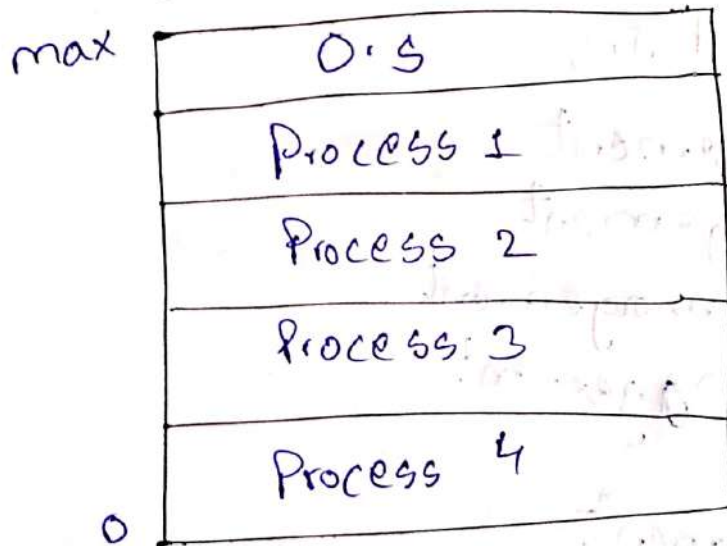
→ process

→ CPU scheduling

→ Swapping

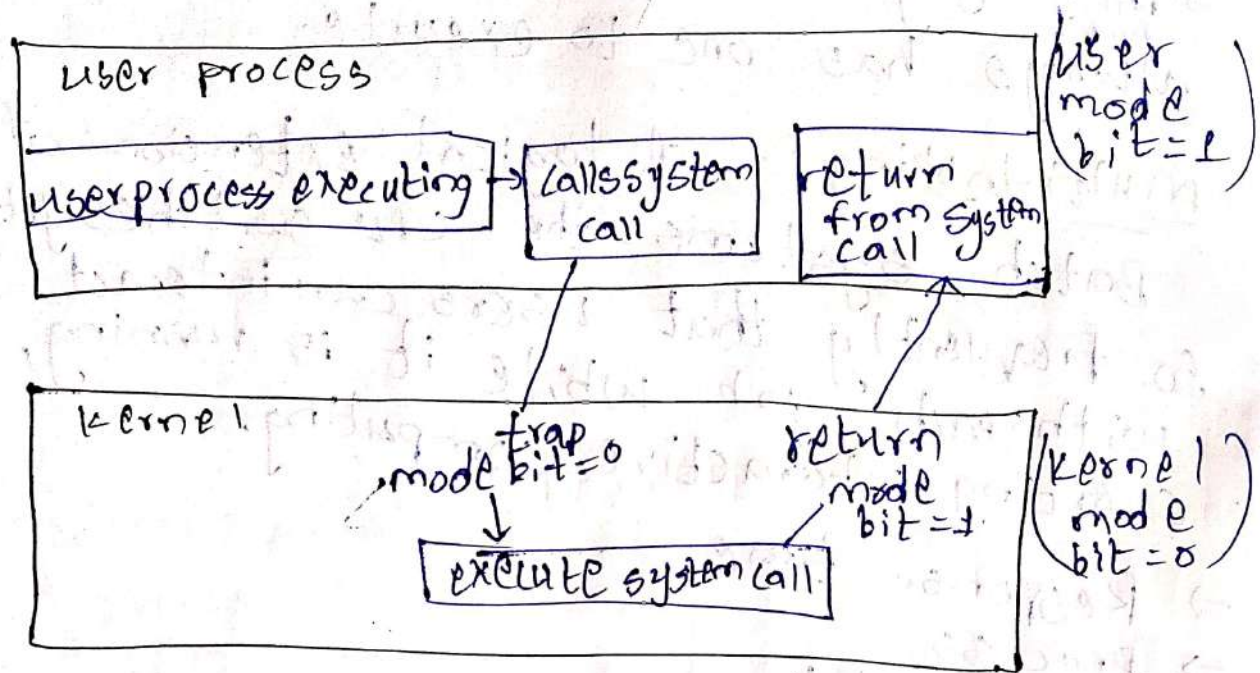
→ Virtual memory

Memory Layout



* Dual-mode operation allows the O.S to protect itself & other system components.

→ User mode & kernel mode.



Timer

- Prevents infinite loop.
- It is set to interrupt the computer after some time period.

Functionalities of OS

- Process Management
- Memory Management
- File system management
- Storage Management
- Caching
- I/O management
- Protection & security.

Process Management

* Executing a program is process. It is a unit of work within the system. is a passive entity, a process

is an active entity.

Resources to accomplish its task.

→ I/O, files, CPU, memory.

→ Initialization data.

* Single threaded process has one program counter specifying the location of the next instruction to execute.

→ Synchronization
→ Communication
→ Handling (deadlock) } O.S. Provides mechanisms for these process.

→ Creating & deleting both user & system processes. } Done by OS

→ Suspending & resuming processes.

* "Deadlock" can be described as multi process

Memory management

→ To execute a program all of the instructions must be in memory.

File system management

* Abstracts physical properties to a logical unit - "file".

→ Primitives

→ Mapping

→ Creating & deleting directories

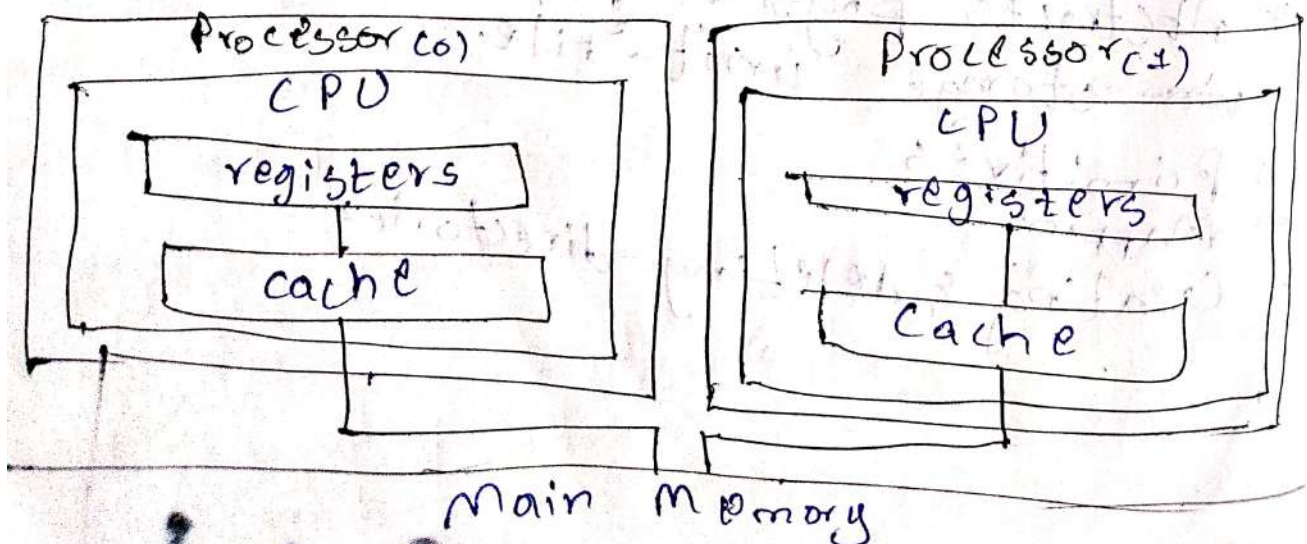
OS activities in storage management

- Mounting & unmounting
- Free-space management
- Storage allocation
- Disk scheduling
- Partitioning
- Protection
- * Buffering (storing data temporarily when it is transferred)
- * The purpose of O.S is to hide the peculiarities of hardware devices from the user.

Distributed Systems

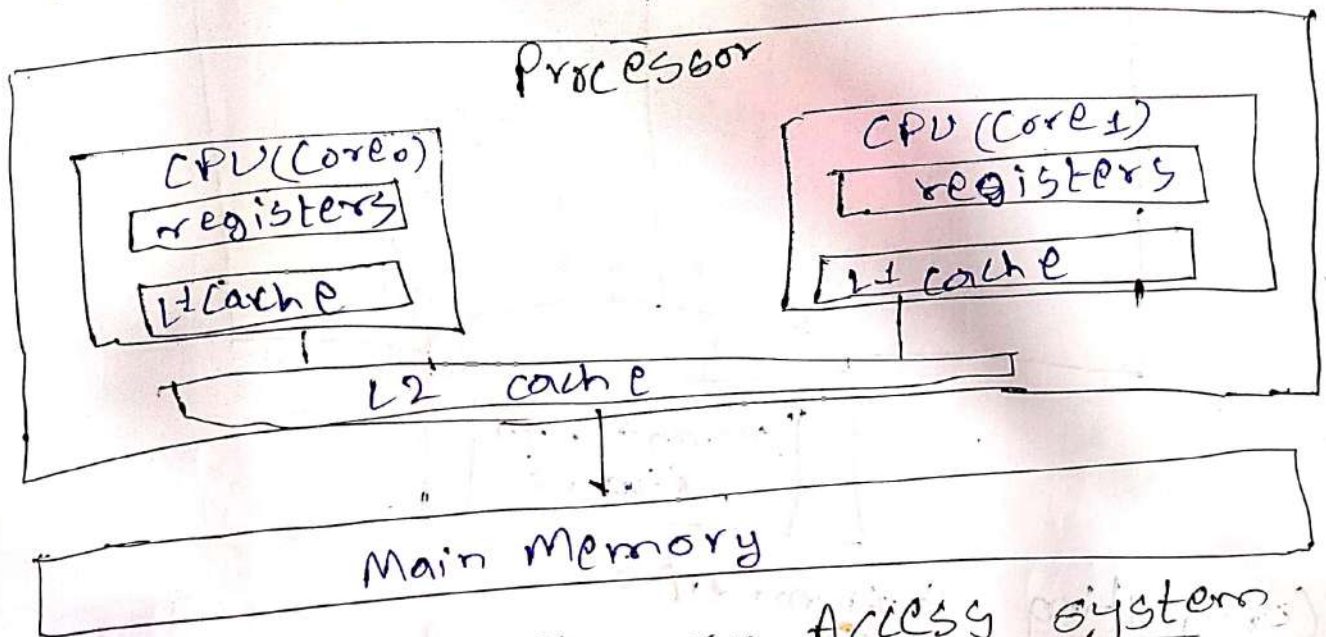
- Local Area Network (LAN)
- Wide Area Network (WAN)
- Metropolitan Area Network (MAN)
- Personal Area Network (PAN)

SYMMETRICAL MULTIPROCESSING ARCHITECTURE

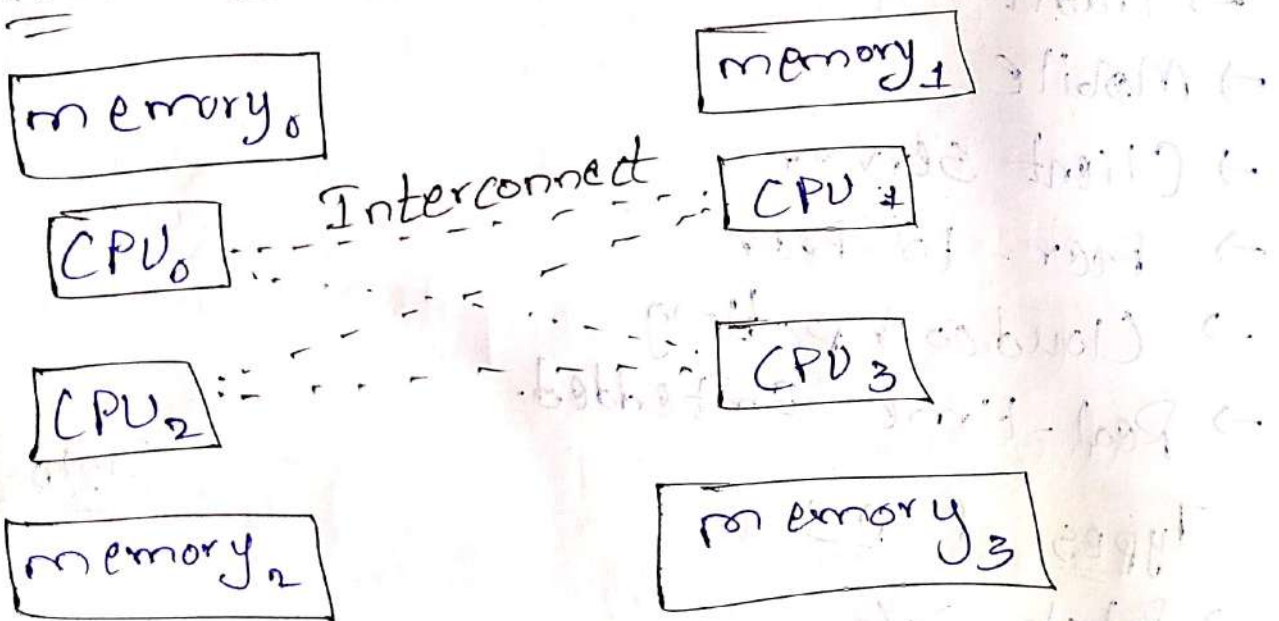


DUAL - CORE DESIGN

* Multichip & multicore

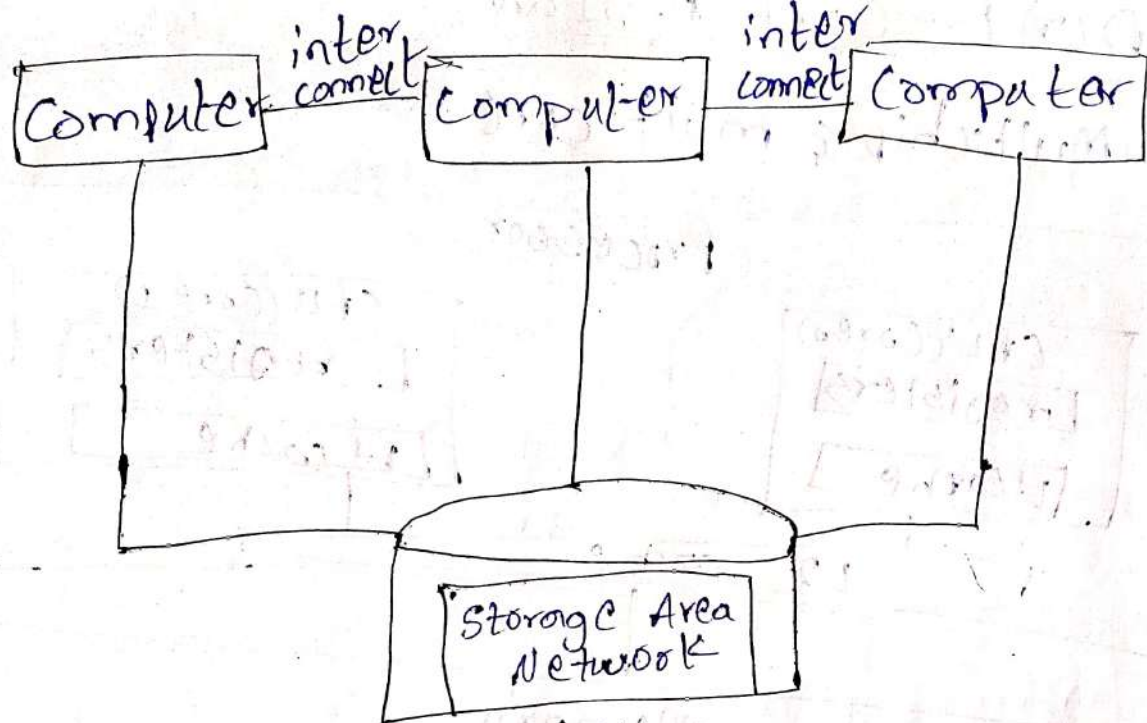


Non-uniform memory Access system



CLUSTERED SYSTEMS

- Multiple systems working together.
- Sharing storage via storage-area-network (SAN).
- * Asymmetric clustering has one machine in hot standby mode.
- * Symmetric clustering has multiple nodes running applications, monitoring each other.



Computing Environments

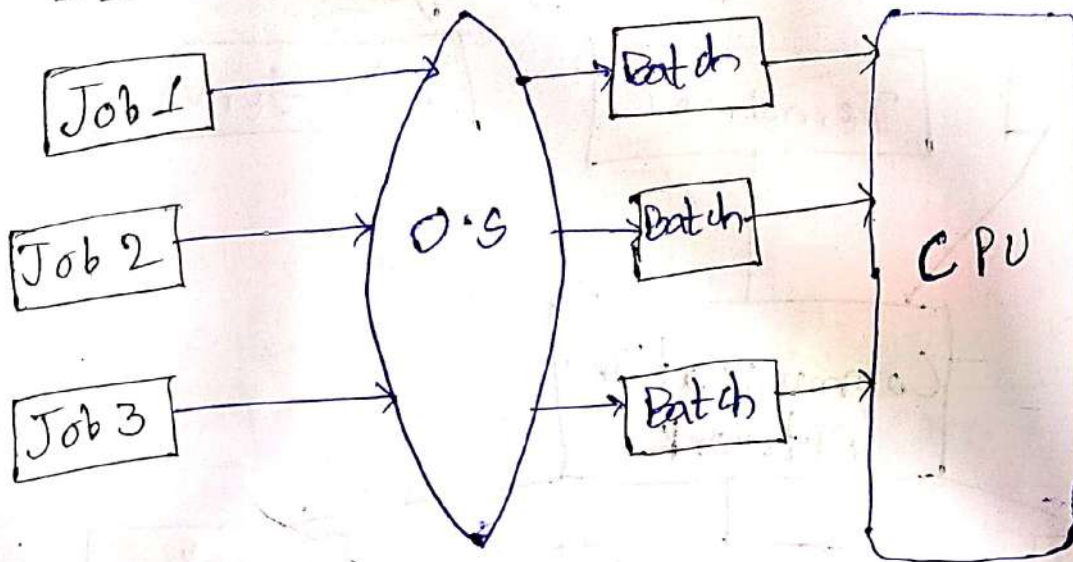
- Traditional
- Mobile
- Client server
- Peer-to-Peer
- Cloud computing
- Real-time Embedded.

Types of O.S

- Batch O.S
- Multiprogramming O.S
- Time Sharing ~~OS~~ O.S
- Multiprocessor O.S
- Real-Time O.S
- Network O.S
- Mobile O.S
- Distributed O.S
- Clustered O.S

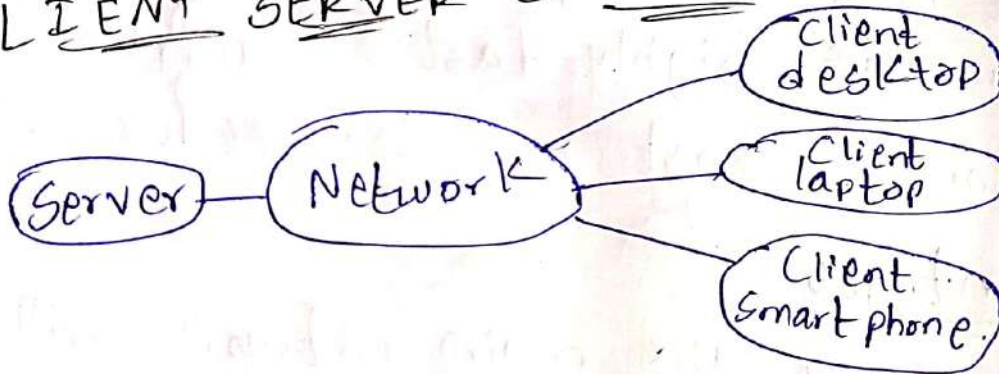
02/08/25

BATCH O.S

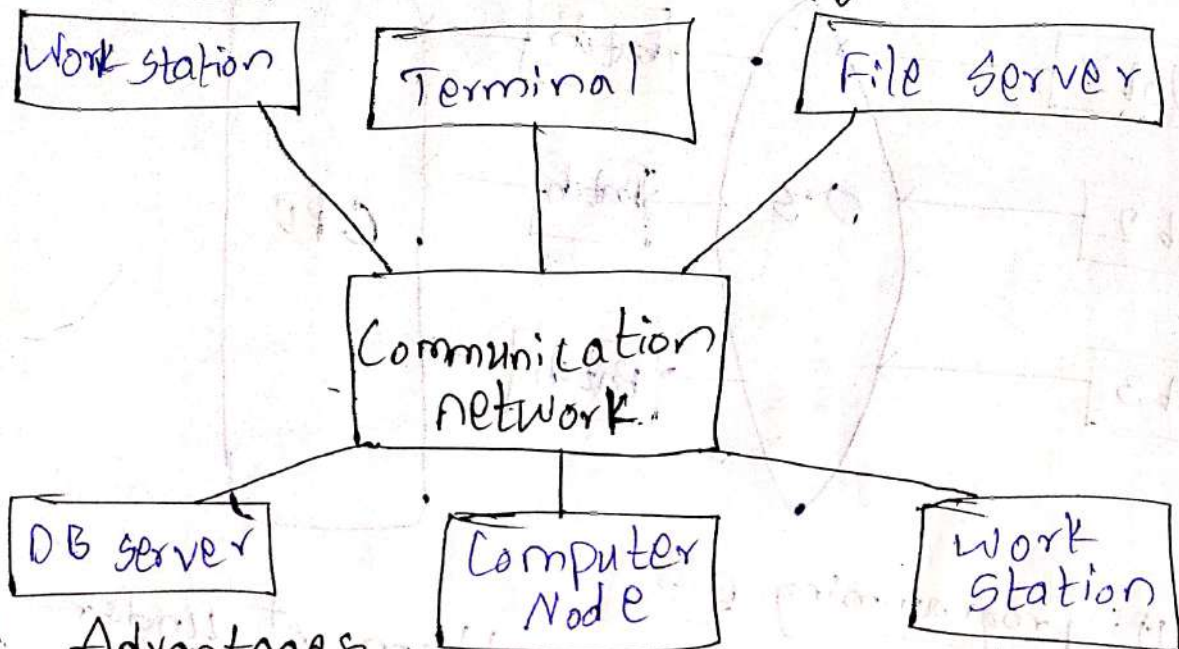


Multi programming O.S
→ To overcome the problem of under-utilization of CPU and main memory, multi programming was introduced.

CLIENT SERVER COMPUTING ENVIRONMENT



DISTRIBUTED operating systems



Advantages

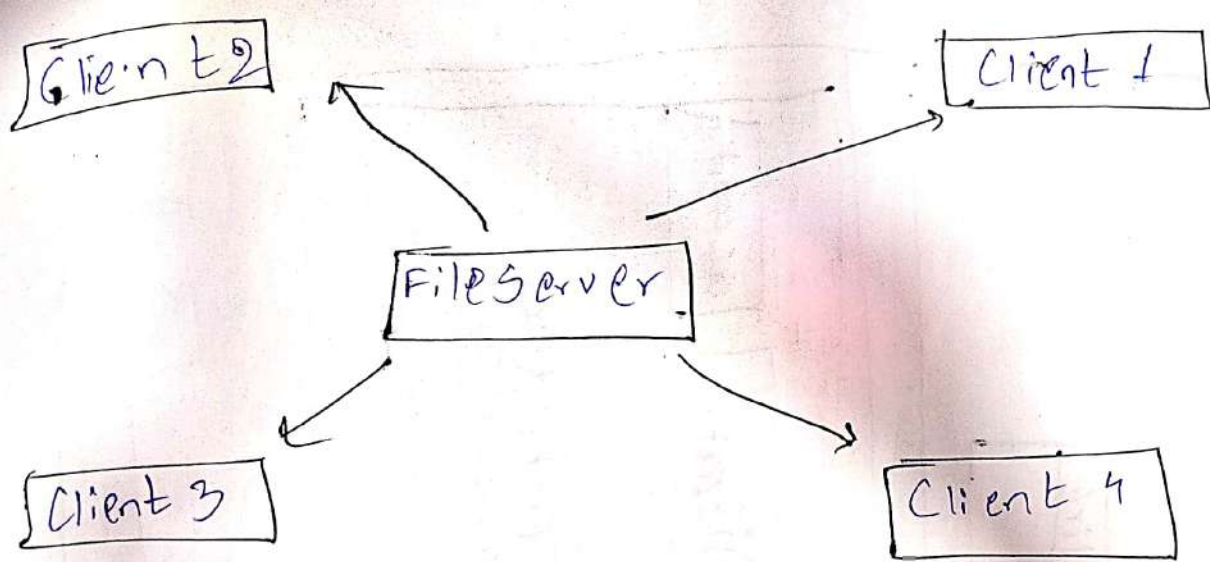
- * Failure of one will not affect the other network communication, as all systems are independent of each other.
- * Since resources are being shared, computation is highly fast & durable.
- * Load on the host computer reduces.

Disadvantages

- * Failure of the main network will stop the entire communication.
- * These are very expensive as they are not readily available.

NETWORK OS

- These systems run on a server & provide the capability to manage data user, groups, applications, and other networking functions.



* This O.S allows shared access to all the files, printers, security, applications, and other networking functions over a small private network.

FREE and OPEN SOURCE O.S

→ O.S are made available in source-code format rather than just binary closed source & proprietary.

EX:- GNU Linux & BSD Unix.

O.S Services

→ Provides an environment for execution of programs & services to programs and users.

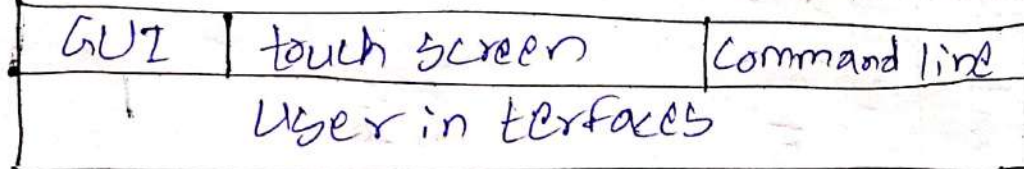
* User interface

* Program execution

* I/O operations

* File system manipulations.

User & other system programs.



System calls

program
execution

I/O
operations

file
systems

communi-
cation

resource
allocation

accounting

error
detection

protection
& security

Services

~~OS~~ OS

Hardware

USER O.S Interface $\Rightarrow$ GUI

* This is a user friendly desktop metaphor interface.

* Many systems are now included both the CLI & GUI interfaces.

Ex:- Windows is a GUI with a CLI "command" shell.

SYSTEM CALLS

* Programming interface to the services provided by the O.S.

* Mostly accessed by programs via a high level Application Programming Interface (API) rather than direct system call use.

Ex:- * Win 32 API for Windows

* POSIX API for POSIX-based systems

* Java API for the Java Virtual Machine (JVM).

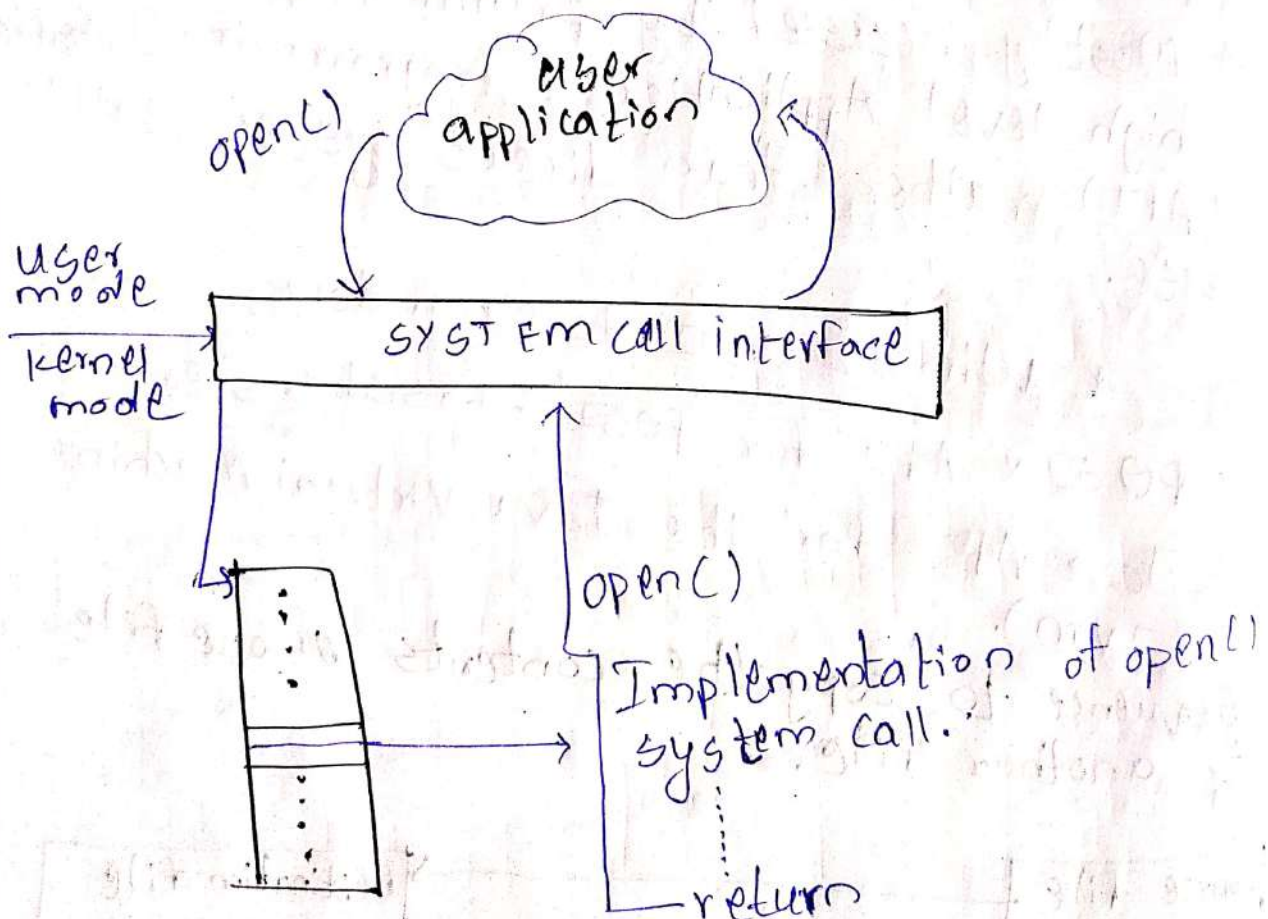
sequence to copy the contents of one file to another file.

Source file

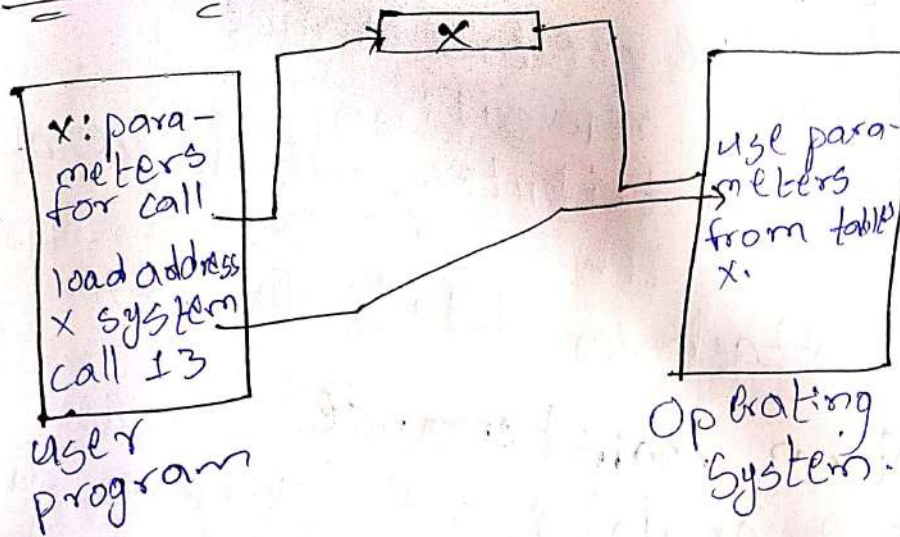
Destination file

Acquire input file name
Write prompt to screen
Accept input
Give output file name
Screen.

Accept Input
Open the input file
If file doesn't exist, abort
Create output file
If file exists, abort
Loop.
Read from input file
Write to output file
Until read fails
Write completion message to screen
Terminate normally.



SYSTEM CALL PARAMETER PASSING



Types of System Calls

- * Process Control
 - create process, terminate process
 - end, abort
 - load, execute
 - get process attributes, set process attributes
 - wait for the time
 - wait event, signal event
 - allocate & free memory.
 - Dump memory if error.
 - Debugger for determining bugs, single-step execution.
 - locks for managing access to shared data b/w processes.

* File management

- Create File, delete File
- Open, close file.
- read, write, reposition.
- get & set file attributes

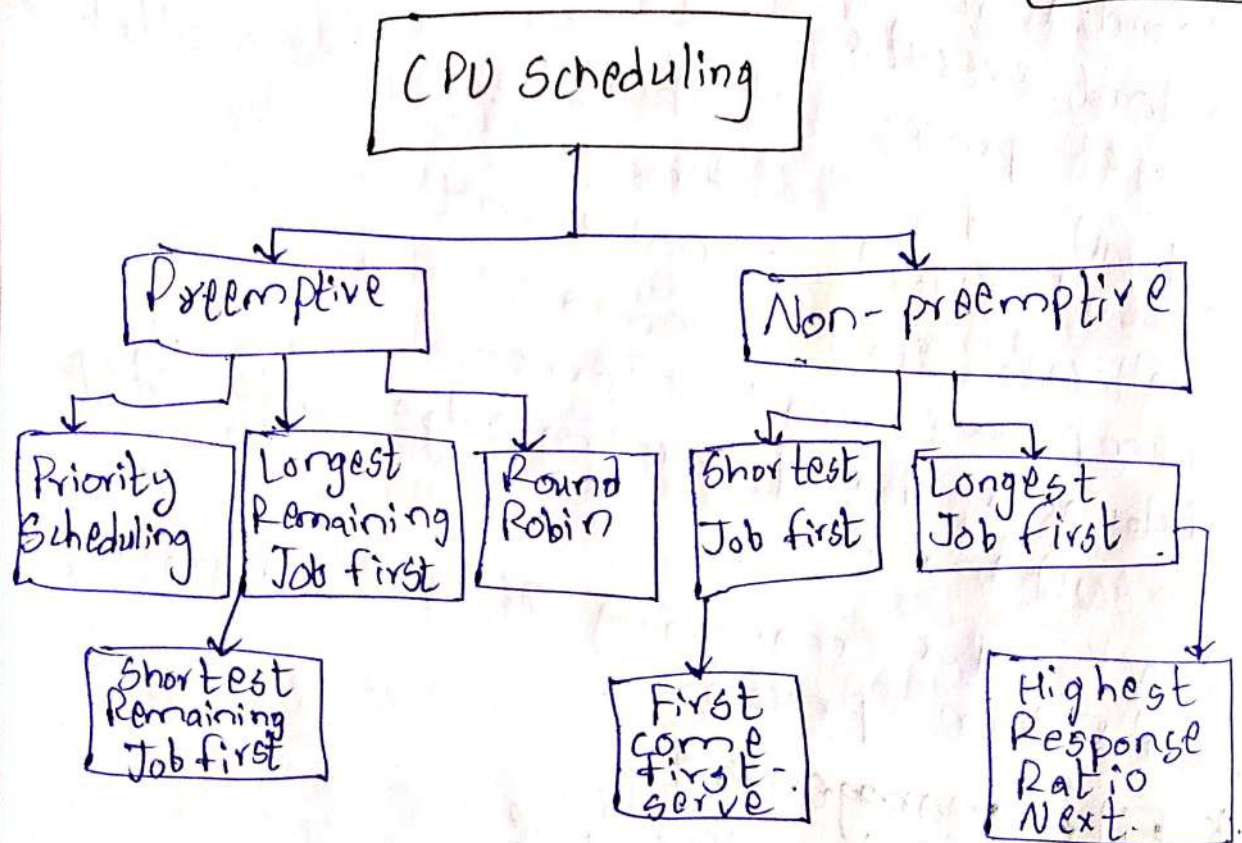
Device Management

- request device, release device
- read, write, reposition.
- get device attributes, set device attributes.
- logically attach or detach devices.

* Information maintenance

- get time or date, set time or date
- get system data, set system data
- get and set process, file, or device attributes.

12/08/25



* In preemptive scheduling, the CPU is allocated to the processes for a limited time.

* In non-preemptive, the CPU is allocated to a process until it gets terminated or switches to a waiting state.

Basic Concepts of Process Scheduling

* This is a cyclic process.

load store
add store
read from file } CPU burst

wait for I/O } I/O burst

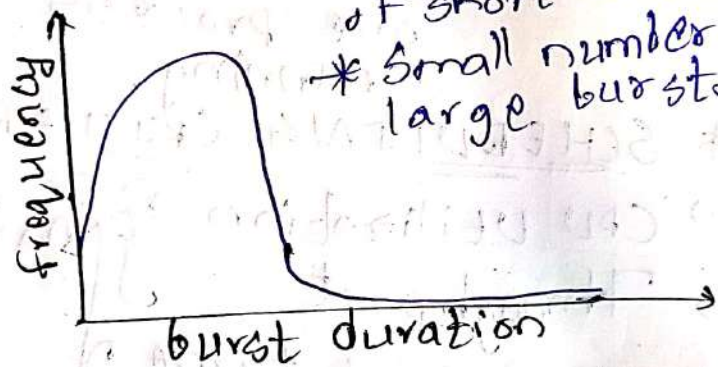
store increment
index
write to file } CPU burst

wait for I/O } I/O burst

load store
add store
read from file } CPU burst

wait for I/O } I/O burst

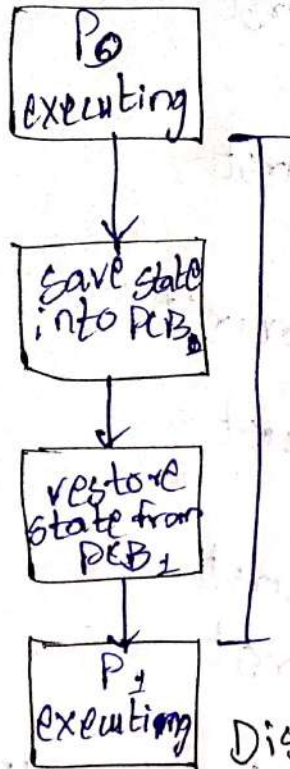
* Large number of short bursts
* Small number of large bursts.



CPU Scheduler

- Short term scheduler selects from among the processes in the ready queue & allocates the CPU to one of them.
- * Switches from running to waiting state.
 - * Switches from running to ready state.
 - * Switches from waiting to ready.
 - * Terminates.

Dispatcher



dispatch latency

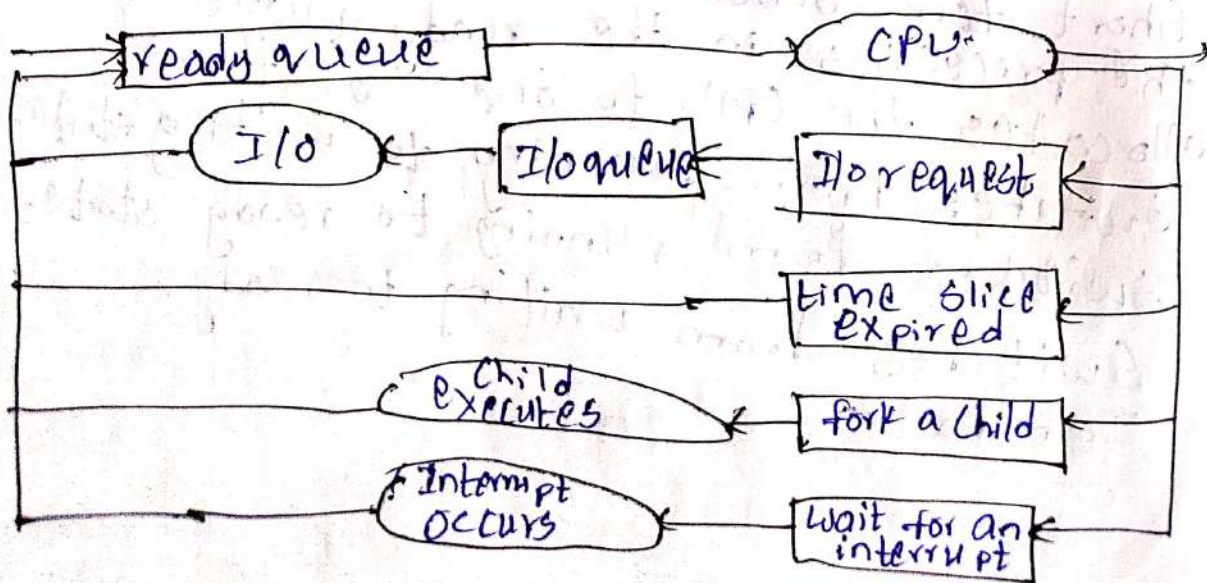
- * switching context
- * Switching to user mode.
- * Jumping to the proper location in the user program to restart the program.

* Dispatcher module gives control of the CPU to process selected by the short-term scheduler.

Dispatch latency = The time it takes for the dispatcher to stop one process & start another running.

* SCHEDULING CRITERIA

- CPU Utilization
 - Throughput
 - Turnaround time
 - Waiting time
 - Response time.
- max } optimization criteria.
min }



* Performance metric: Turnaround time

$$T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}}$$

First - IN - First - out (FIFO)

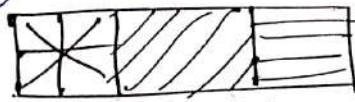
* First come, first served (FCFS)

• Very simple & easy to implement

Ex:- A arrived first before B which arrived just before C.

* Each Job runs for 10 seconds.

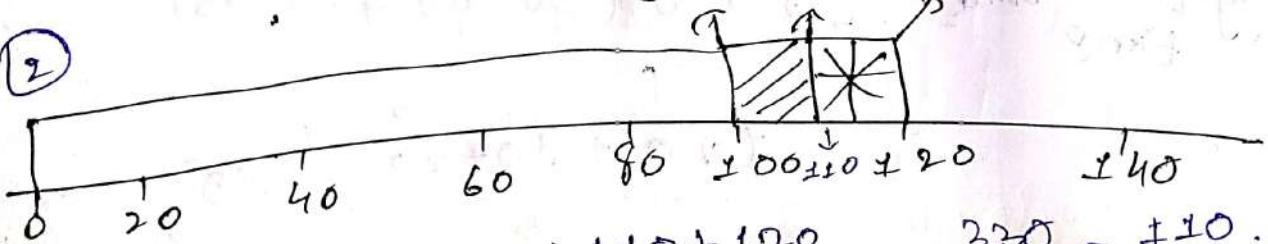
①



$$\text{Avg turnaround time} = \frac{10 + 20 + 30}{3} = \frac{60}{3} = 20$$

* Due to convoy effect large number of short potential consumers fall behind the heavy-weight resource consumer

②

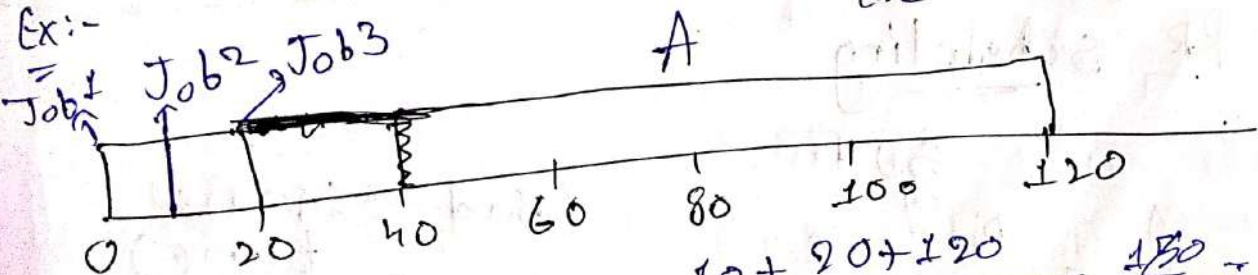


$$\text{Avg turnaround time} = \frac{100 + 110 + 120}{3} = \frac{330}{3} = 110$$

Shortest Job First (SJF)

* runs the shortest job first then the next shortest.

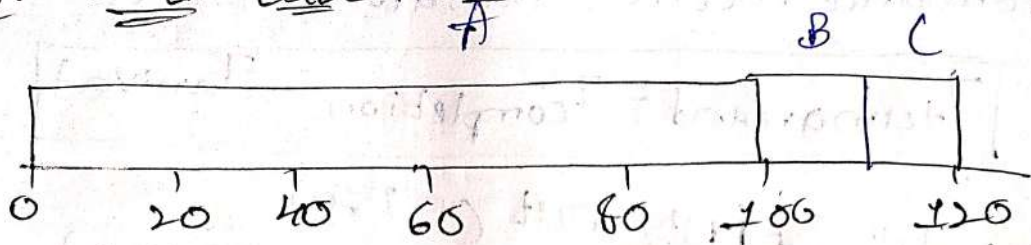
Ex:-



$$\text{Avg turnaround time} = \frac{10 + 20 + 120}{3} = \frac{150}{3} = 50$$

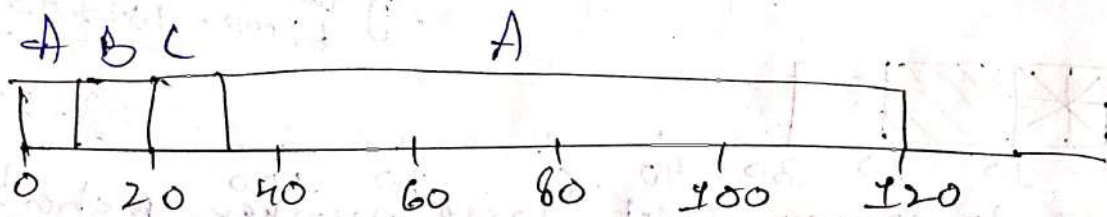
* Compared to both algorithms SJF is more efficient than FIFO.

SJF with Late Arrivals



$$\text{Avg turnaround} = \frac{100 + (110 - 10) + (120 - 10)}{3} = 103.33 \text{ sec.}$$

Shortest Time to completion First (STCF)



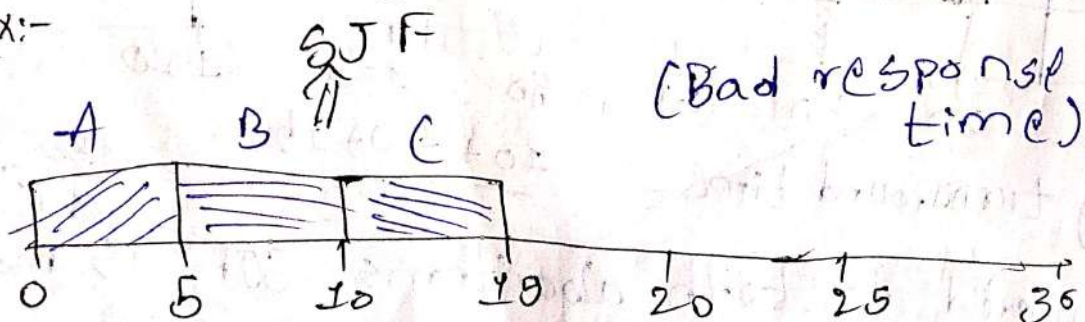
$$\text{Avg turnaround time} = \frac{(120 - 0) + (20 - 10) + (30 - 10)}{3} = \frac{120 + 10 + 20}{3} = \frac{150}{3} = 50 \text{ sec.}$$

* New scheduling metric: Response Time

$$\boxed{\text{Response} = T_{\text{first run}} - T_{\text{arrival}}}$$

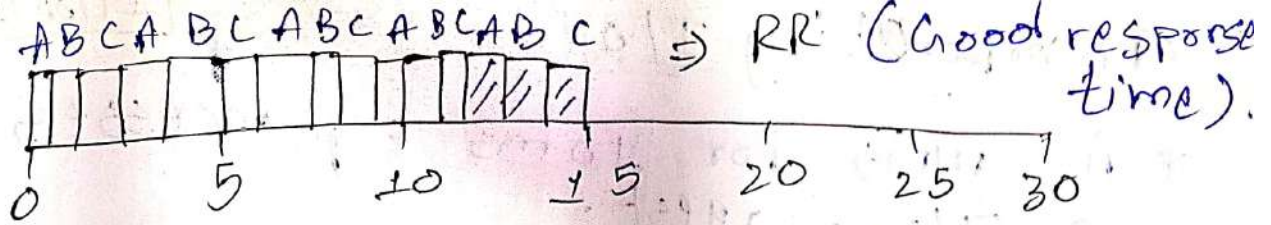
RR scheduling

Ex:-



(Bad response time).

$$T_{\text{avg response}} = \frac{0 + 5 + 10}{3} = 5 \text{ sec.}$$

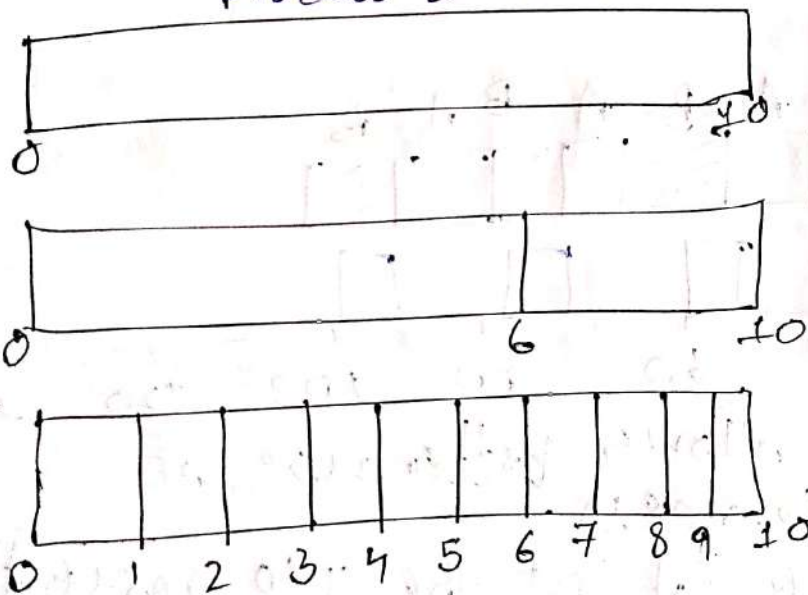


$\therefore$ avg response = $\frac{0+1+2}{3} = \frac{3}{3} = 1 \text{ sec.}$

* In round robin we have to check where the task is completed & find the turn around time. For example in the above problem A-13, B-14, C-15 got completed

Avg Turnaround Time = $\frac{13+14+15}{3} = \frac{42}{3} = 14 \text{ sec.}$

Time Quantum & Context Switch Time
Process time = 10



quantum	Context Switches
12	0
6	1
1	9

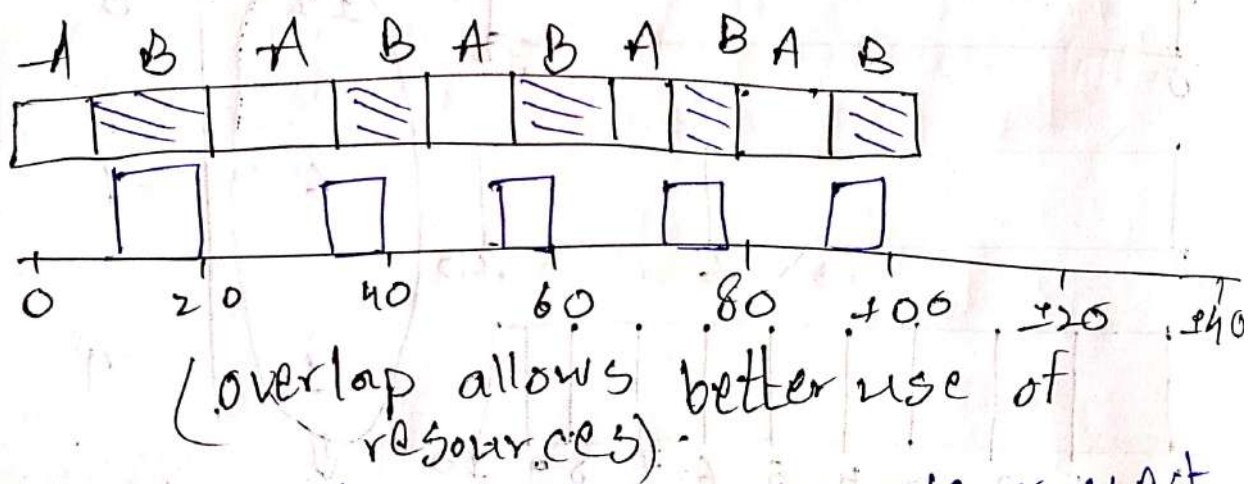
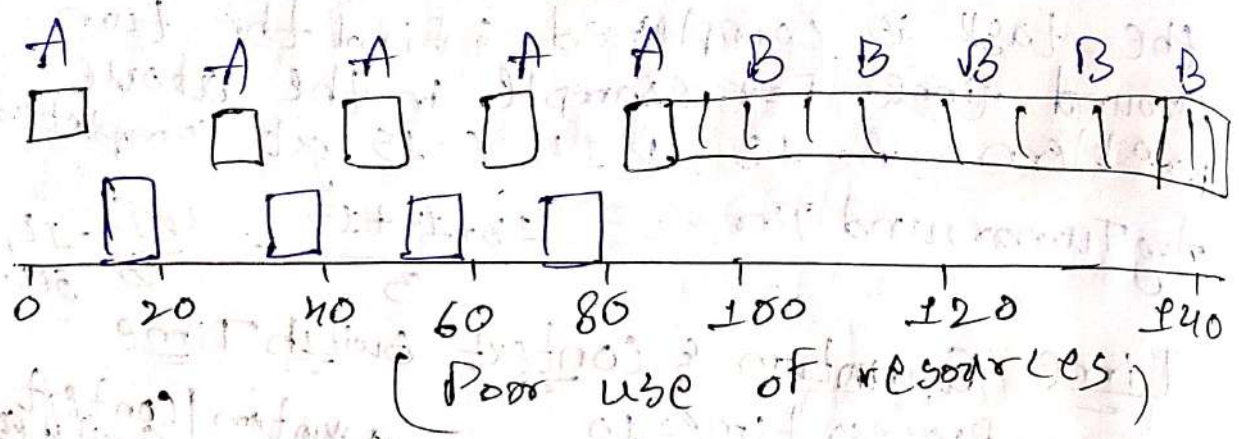
in seconds.

- * In first one quantum i.e; time is 12 sec not possible so no context switches
- * In second one quantum is 6 one context switch is possible.
- * In 3rd quantum is one so, a are possible.

Incorporating I/O

* A run for 10ms & processes an I/O request.

* A & B needs 50ms.



* In the time of 'B' the I/O request is processed. This is done in less time compared to above process.

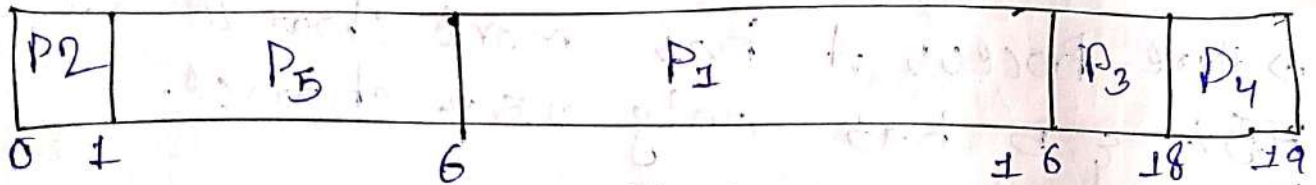
Priority Scheduling

* A priority number is associated with each process.

* The CPU is allocated to the process with the highest priority (smallest value highest priority).

Ex ⁿ	Process	Burst Time	Priority
	P ₁	10	3
	P ₂	1	1
	P ₃	2	4
	P ₄	1	5
	P ₅	5	2

* Priority Scheduling - Gantt chart

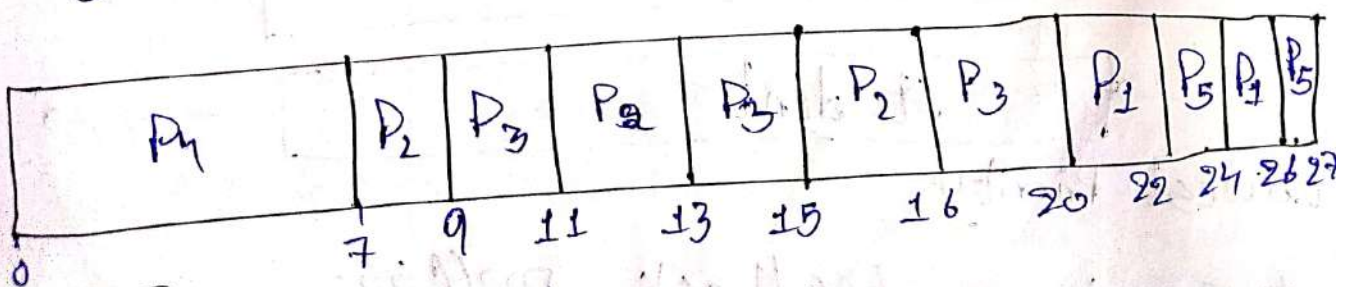


Average waiting time = 8.2

* Priority scheduling with Round-Robin

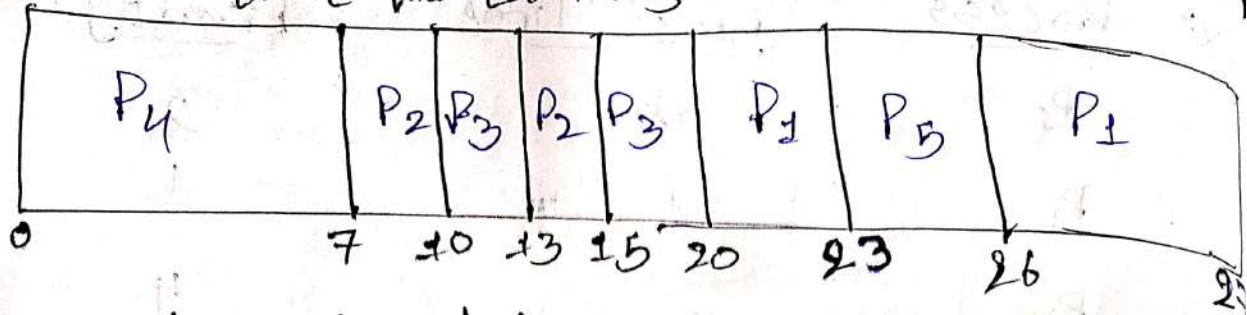
Process : Burst Time : Priority Gantt chart

P ₁	4	3	* Time quantum = 2
P ₂	5	2	
P ₃	8	2	
P ₄	7	1	
P ₅	3	3	



Gantt Chart.

* If time quantum = 3



Lottery Scheduling

* The scheduler picks a winning ticket

Ex:- There are 100 tickets.

* Process A has 75 tickets: 0 - 74

* Process B has 25 tickets: 75 - 99

→ Here Process A has more chance i.e., 75% & B has only 25% chance.

Multilevel Queue Scheduling

highest Priority

→ System Processes →

→ Interactive →

→ Interactive Editing →

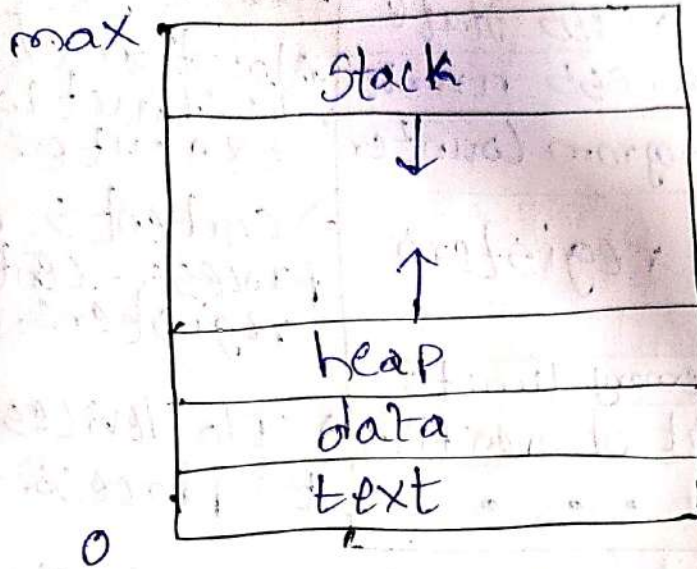
→ Batch →

→ student →

Lowest Priority

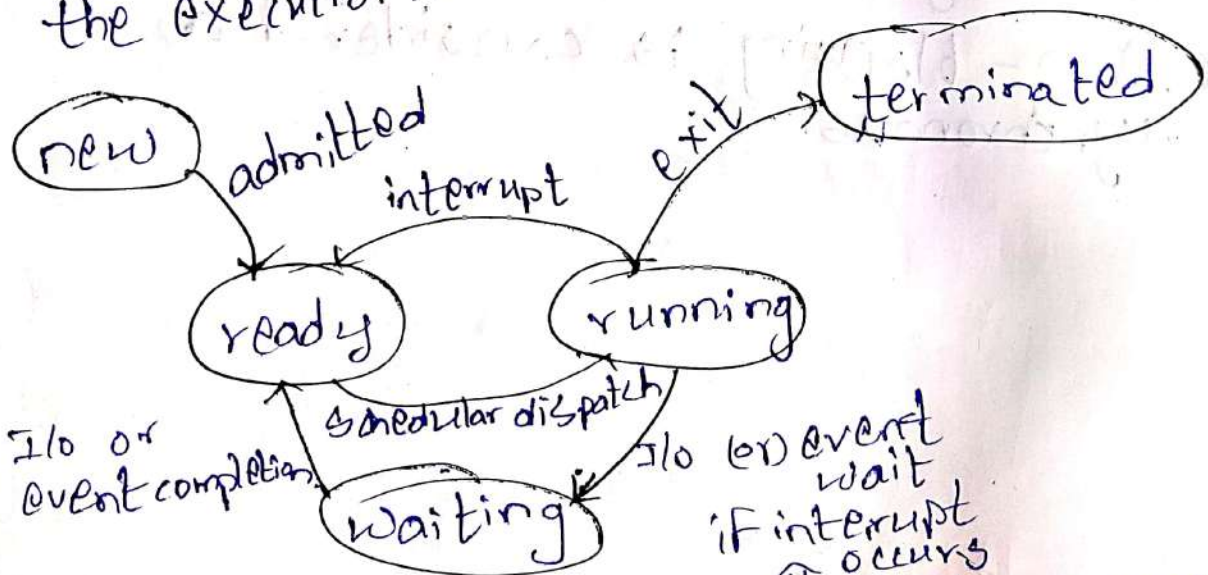
* This is a feedback process.

Process in memory



PROCESS STATE

- New ⇒ Process is being created.
- Running ⇒ Instructions are being executed.
- Waiting ⇒ Process is waiting for some event to occur.
- Ready ⇒ waiting to be assigned to a processor.
- Terminated ⇒ The process has finished the execution.



⇒ new → ready → running → ready → running → terminated.
↓
Asks for correct input if correct input is given then process is terminated.

Process Control Block (PCB)

memory allocated to the process $\Leftarrow$	Process state	$\Rightarrow$ running, waiting
	Process number	$\Rightarrow$ location of next instruction to execute.
	Program Counter	
	registers	$\Rightarrow$ contents of all process-centric registers.
	memory limits	
	list of open files	$\Rightarrow$ I/O devices allocated to process.
	...	

* This is the information associated with each process (task control block)

SYNCHRONIZATION

* Coordination of execution of multiple processes in a multi process system.

* Blocking is considered synchronous.

* Non-blocking is considered as asynchronous.

Remote Procedure Calls

→ Remote procedure calls abstracts procedure calls between processes on networked systems.

* Stubs - client-side proxy for the actual procedure on the server.

* The client-side stub locates the marshal^{server} the parameters.

* MIDL (Microsoft Interface Definition Language). stub code compiles from this

- Readers - Writers
- Reader - Only read the data set; they do not perform any updates.
 - Writers - Can both read & write.

* Structure of a reader process

```
while (true) {  
    wait {mutex};  
    read count++;  
    if (read - count == 1)  
        wait (rw_mutex);  
    signal (mutex);  
    wait (mutex);  
    read count--;  
    if (read - count == 0)  
        signal (rw_mutex);  
    signal (mutex);  
}
```

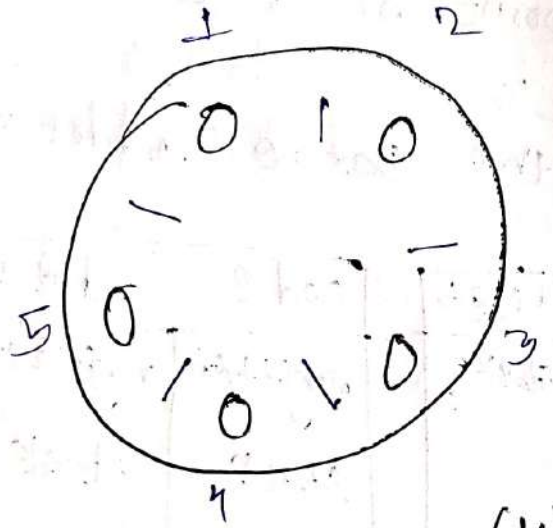
* Structure of a writer process

```
while (true) {  
    wait (rw_mutex);  
    signal (rw_mutex);  
}
```

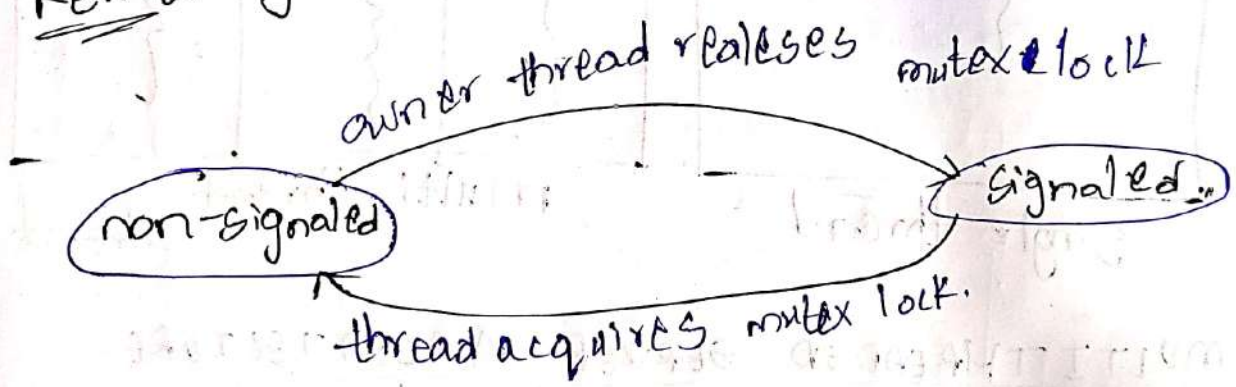
Dining - Philosophers Problem

- 'N' philosophers sit at a round table.
- They spend their lives in alternating thinking & eating.
- They do not interact with their neighbors.

- They pick two chopsticks to eat from a rice bowl.
- In the case of 5 philosophers, the shared data.
 - * Bowl of rice
 - * Semaphore chopstick [5].



Kernel Synchronization (Windows)



Thread

- * A thread is a lightweight compared to processes.
- * It is a smallest unit of execution that an O.S can schedule & run.
- * A process is a complete program & a thread is like a worker in that program.

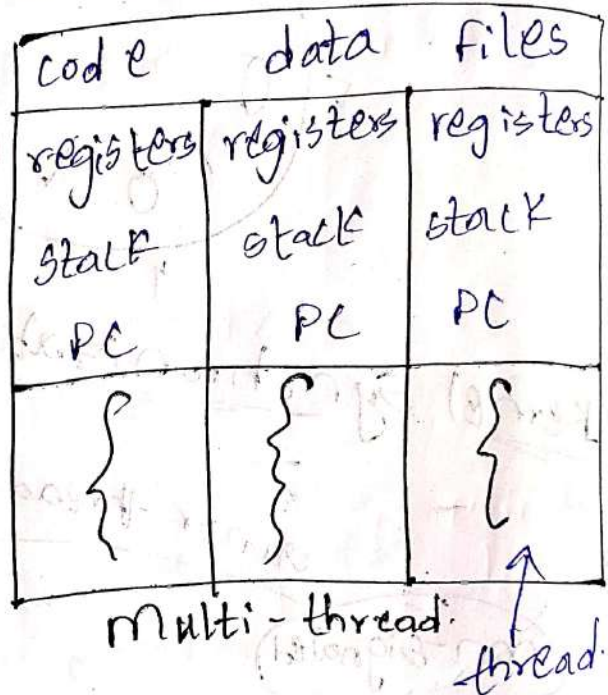
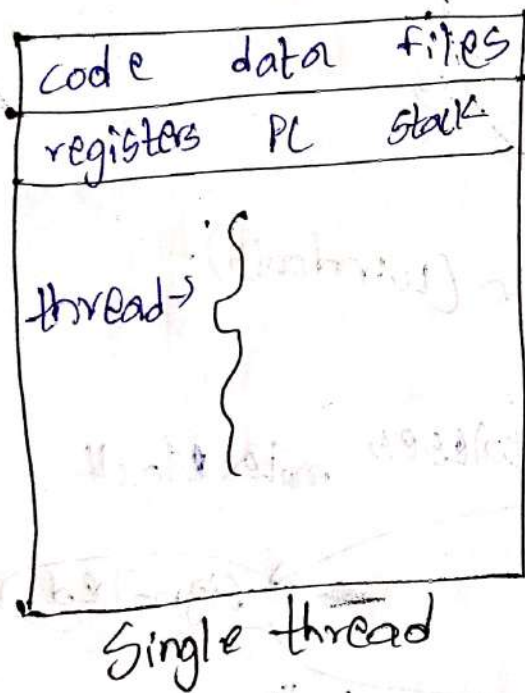
* A process can have one thread (single threaded) or many threads (multi-threaded).

Multi-threaded Program

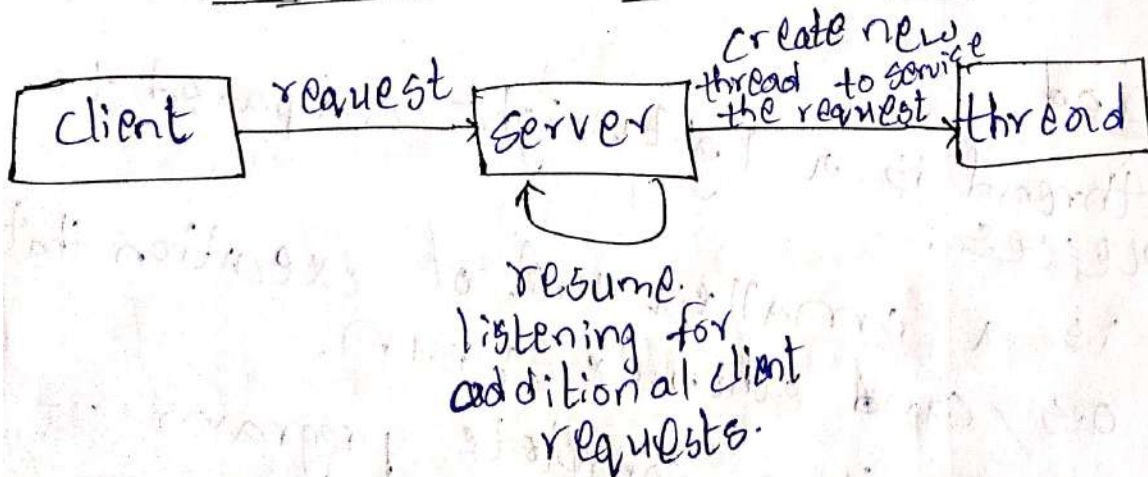
Multi-threaded Program
→ A multi-threaded program has more than one point of execution.

→ Multiple PC's

- Multiple PC's
- They share the same address space.



MULTITHREADED SERVER ARCHITECTURE



Benefits

- Responsiveness
- Resource Sharing
- Economy
- Scalability.

* Multicore or multi processor systems puts pressure on programmers.

Challenges include:

- Dividing activities
- Balance
- Data Splitting
- Data Dependancy
- Testing & debugging.

* Parallelism $\leftarrow$ data (distributes subsets of the same data across multiple cores).
task (distributes threads across cores)

→ implies a system can perform more than one task simultaneously.

* Concurrency

→ Supports more than one task making progress.

AMDAHL'S LAW

$$\text{Speedup} \leq \frac{1}{S + \frac{(1-S)}{N}}$$

S = Serial portion

N = processing cores;

User Threads

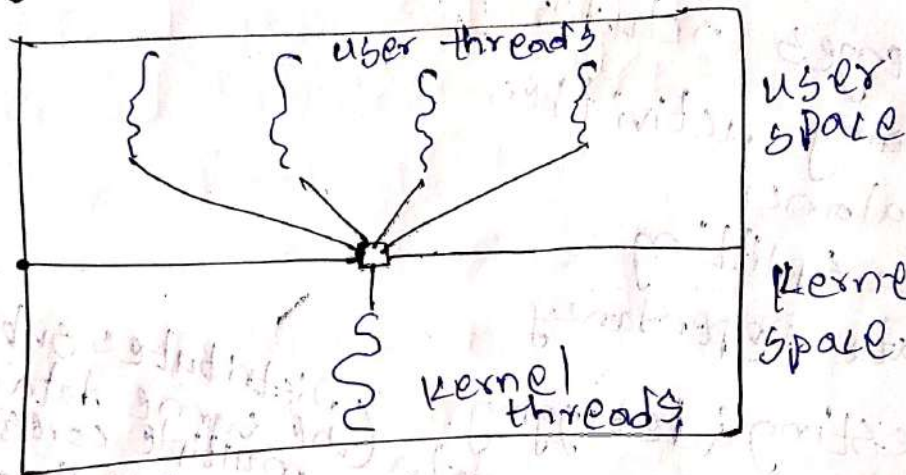
management done by user-level threads library.

- POSIX pthreads
- Windows threads
- Java threads.

Kernel threads → supported by kernel.

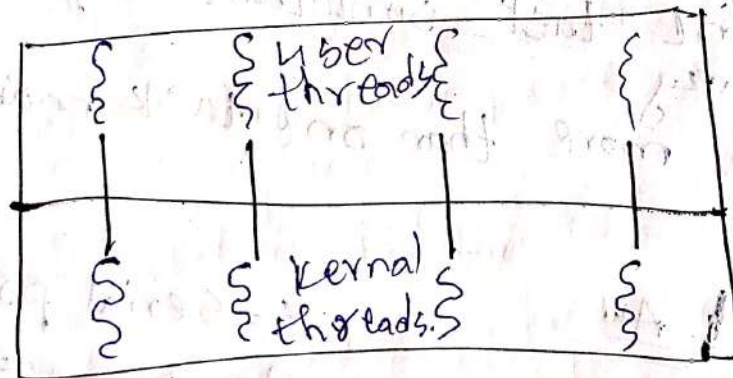
- Windows
- Linux
- Mac OS
- Android
- iOS.

Many-to-one Ex:- Solaris Green Threads.

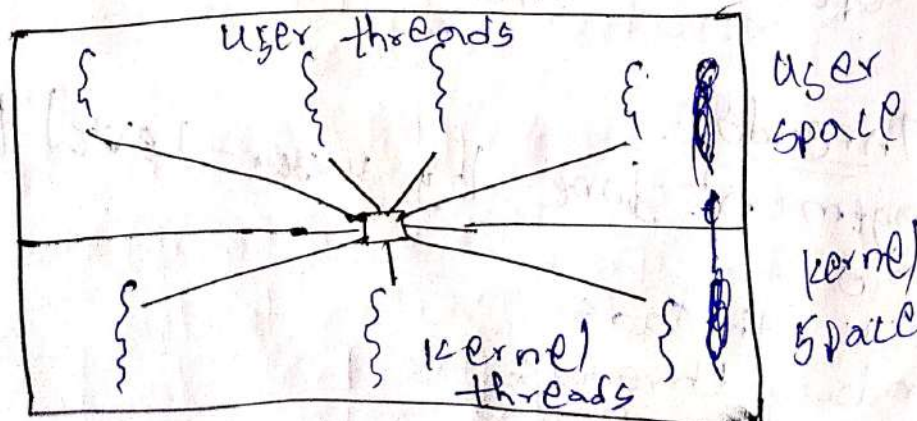


One-to-one

Ex:- Windows
Linux



Many-to-Many



Peterson's Algorithm

```
while (true) {  
    flag[i] = true;  
    turn = j;  
    while (flag[j] && turn == j);  
    flag[i] = false;  
}
```

⇒ For process P_i :

Concurrency Problems

- Non-Dead Lock Bugs
- Deadlock Bugs.

* Non-deadlock bugs make up a majority of concurrency bugs, according to Lu's study.

⇒ Two major types of non deadlock bugs

- (a) Atomicity Violation
- (b) Order violation.

* To fix Atomicity Violation, simply add locks around the shared-variable reference.

Deadlock bugs

* Complex locking protocols is known as deadlock.

Ex- Thread 1 is holding Lock 1. & waiting for L2 to be released, but L2 is waiting for the release of L1 so

neither can be released and can run where a dead lock occurs.

Ex:- Thread 1:

pthread_mutex_lock(L1);

pthread_mutex_lock(L2);

Thread 2:

pthread_mutex_lock(L2);

pthread_mutex_lock(L1);

Synchronization Tools

- Locks
- Condition Variables
- Mutex
- Semaphores
- Monitors.

Race Condition:- This occurs when a multiple threads or processes ^{access} shared resource simultaneously.

Critical Section Problem

* Every process has critical section segment of code. Here the shared resource is accessed (or) modified in a multi-threaded system.

General structure of process

```
while (true) {  
    entry section  
    critical section  
    exit section  
    remainder section.  
}
```


Solution to Critical-section problem

- ① Mutual Exclusion $\Rightarrow$ If a process P_i is executing in critical section then no other process can execute in their critical sections.
- ② Progress $\Rightarrow$ If no process is executing in the critical section then the processes that are waiting to the critical section should have to be ^{enter} processed without any delay.
- ③ Bounded Waiting
There should be a limit on how many times other processes can enter their critical sections before a waiting process gets its turn.

* For making Peterson's solution work correctly on modern computer architecture we must use memory barrier.

Memory Barrier

* This is an instruction that forces any change in memory to be made visible to all other processors.

Memory models are two types:-

- ① Strongly-ordered $\Rightarrow$ memory modification of one process is visible to all other processors.

- ② Weakly ordered: where memory modification of one processor may not be immediately visible to all other processors.

Synchronization Hardware

Many systems provide hardware support for implementing the critical section code.

Forms of hardware support

① Hardware Instructions

- * Test-and-set instruction
- * Compare-and-swap instruction.

② Atomic Variables

- * This provides atomic (uninterruptable) updates on basic data types such as integers & booleans.

③ Mutex Locks

The simplest software tool to solve CS problem.

Syntax →

```
while (true) {  
    acquire lock  
    critical section  
    release lock  
    remainder section  
}
```

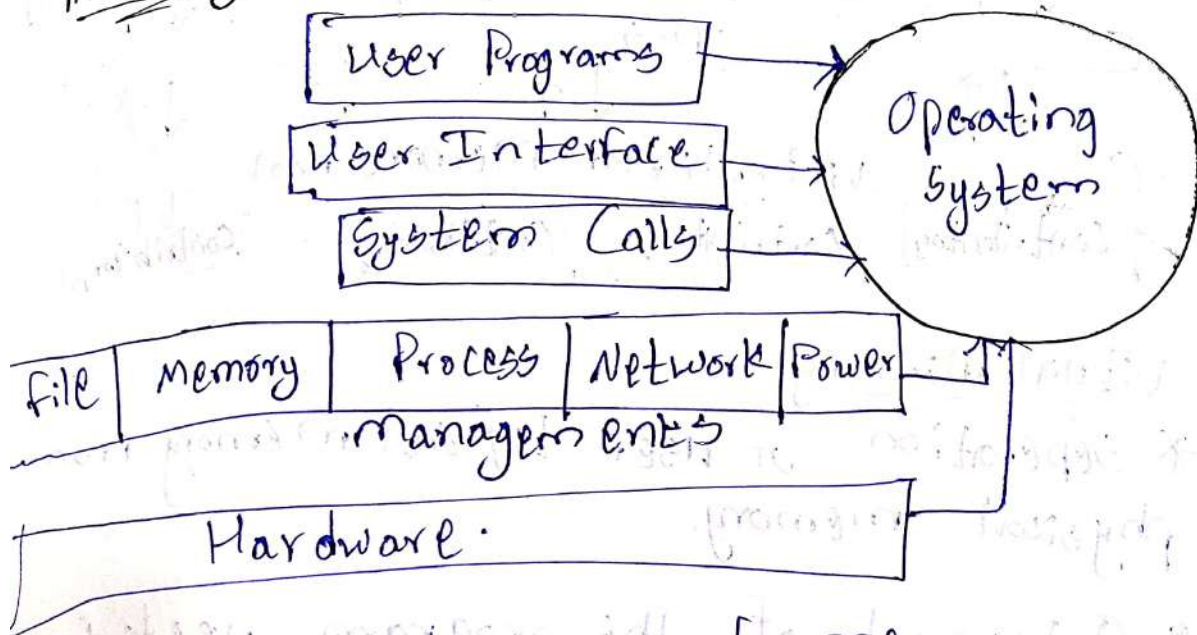

Semaphore

- Synchronization tool that provides more sophisticated ways (than mutex locks) for processes to synchronize their activities.
- * S - integer variable.
- * Can only be accessed via two indivisible (atomic) operations
 - wait() & signal().
- Counting semaphore ⇒ integer value can range over unrestricted domain.
- Binary semaphore ⇒ integer value can range only between 0 & 1.

Monitors

- A high level abstraction that provides a convenient & effective mechanism for process synchronization.
- * Only one processor may be active within the monitor at a time.

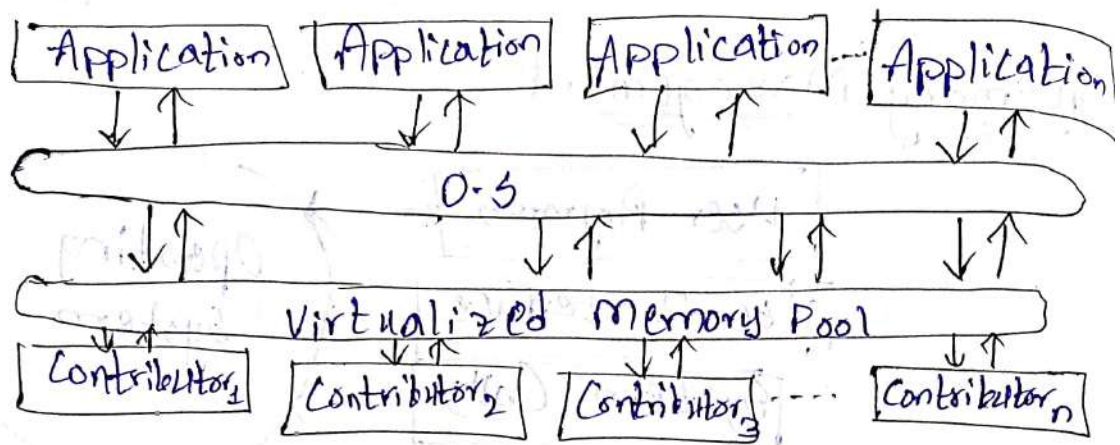
Memory Management



- * This is the process of memory management related O.S.
- * Memory management is one of the sub-process in O.S working process.

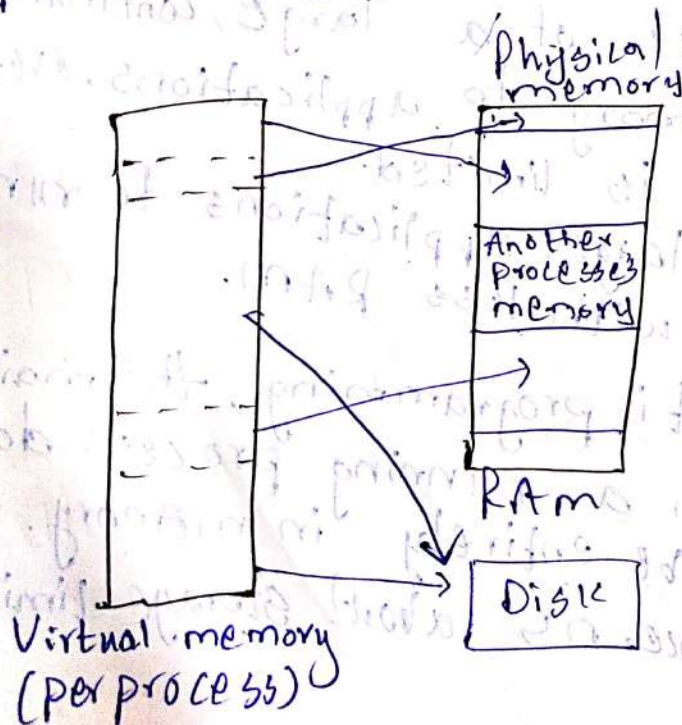
Memory Virtualization

- A technique used by O.S to give the appearance of a large, continuous block of memory to applications, even if the RAM is limited.
- * This allows larger applications to run on systems with less RAM.
- * Supports multi programming, the main advantage is, a running process does not need to be entirely in memory.
- * Eliminates concerns about storage limitations.



Virtual Memory

- * Separation of user logical memory from physical memory.
- * Only part of the program needs to be in memory for execution.
- * Logical address space > Physical address space
- * Allows address spaces to be shared by ~~any~~ several processors also for efficient process creation.



Memory API :-

- * Includes stack and heap memory
- * Memory allocation, deallocation & the diff b/w stack & heap memory.

Types

1) Stack memory (LIFO)

```
void fun CL)
```

```
{
```

```
int a = 10;
```

```
}
```

⇒ Declares a variable in stack memory.

- * Very fast compared to heap memory.

- * Has a limited size.

- * Used to store local variables and function calls.

2) Heap

- * No size limit

- * Allocated during runtime

- * Variables are accessed globally.

```
int *a = (int *) malloc (sizeof(int));
```

↓
Declaration of variable in heap memory.

⇒ Free (-)

```
int *a = (int *) malloc (int);
```

free(a); ⇒ removes the memory or deallocates it.

ADDRESS TRANSLATION

16/09/25

1) Virtual Memory

2) Physical Memory (logical)

* Virtual address memory is mapped to Physical address memory.

→ Dynamic Relocation with "base" and "bounds" registers.

* The base register sets the starting physical ~~mem~~ address for a process's memory space, translating each virtual address as (physical address = virtual address + base).

* The bounds register limits the address space size, raising an exception if a virtual address is out of bounds.

→ Interposition in Address Translation:

* Inserting control between actions, is central to addressing translation.

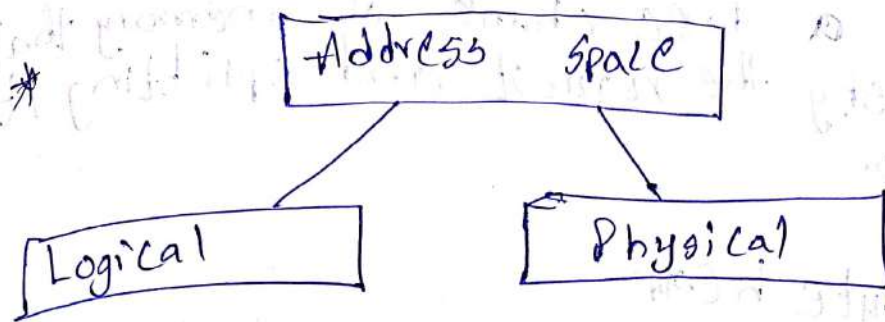
* Interposition provides transparency

PAGING

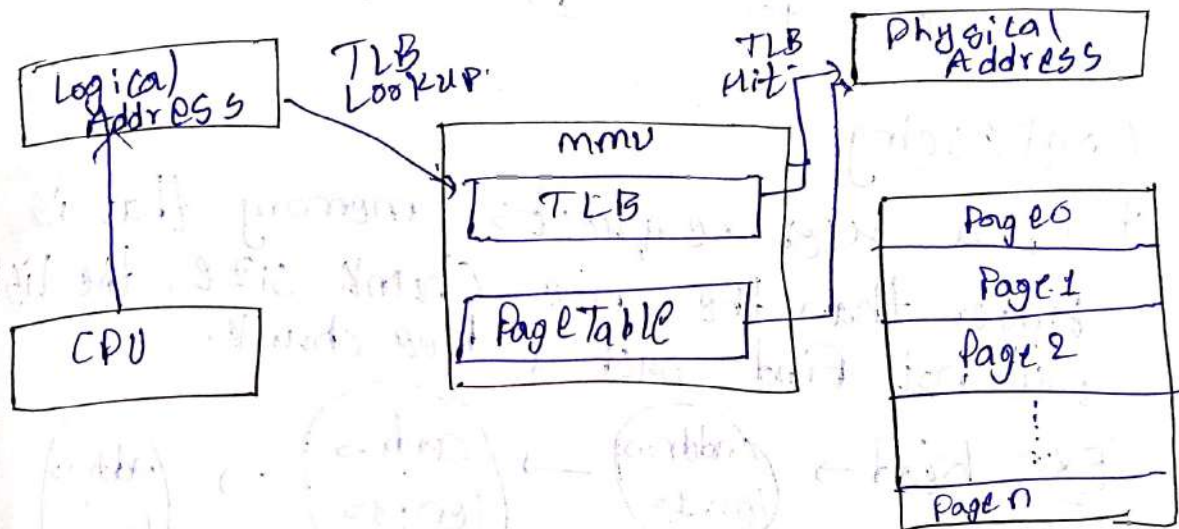
* Address translation in paging in O.S is an address space that is the range of valid addresses available in a program.

* It is the memory space accessible to the program.

* The memory can be physical or virtual.



ADDRESS TRANSLATION WITH MMU



Free Space Management

* A File system is responsible for allocating the free blocks to the file; therefore it has to keep track of all the free blocks present on the disk.

* Two Approaches:-

1) Bit Vector

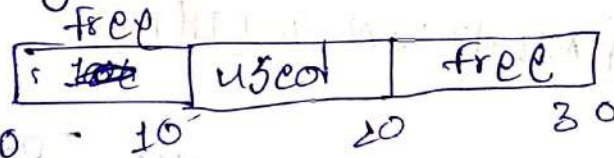
* Free space list is implemented as a bit map vector. Contains number of bits.

- Each bit = One block.
- if block = empty (bit = 1)
- if block = full (bit = 0)

SPLITTING

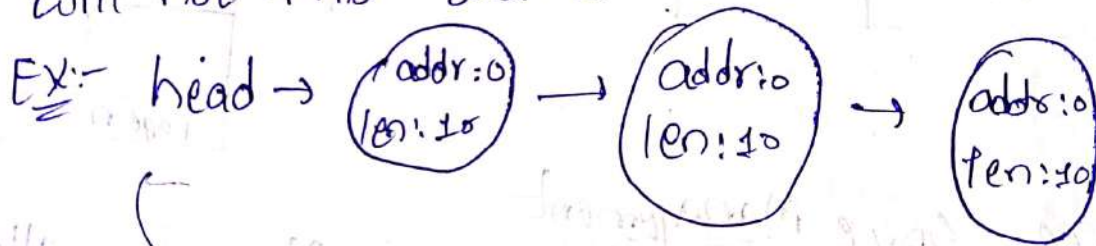
* Finding a free chunk of memory that can satisfy the request and splitting it into two.

Ex:- 30 byte heap



Coalescing

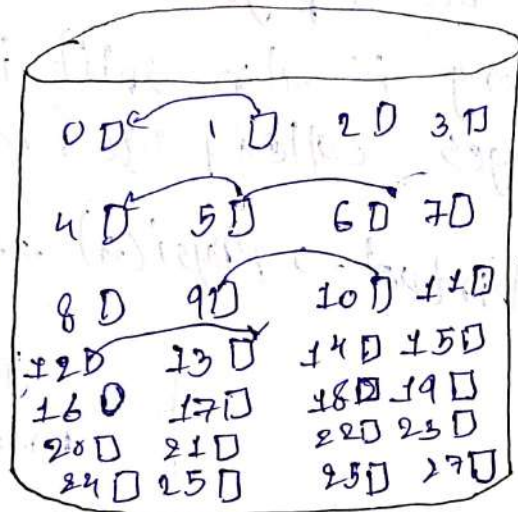
* If a user requests memory that is bigger than the free chunk size, the list will not find such a free chunk.



Coalescing
free chunks.

Note:- `FREE()` is the method used to deallocate or release the memory from the space.

Linked Free Space List On Disk



CPU Virtualization

- * Enables single physical CPU to function as multiple virtual CPUs.
- * Allows multiple applications to run simultaneously on a single physical machine.
- * This works particularly in cloud environments.
- * Managed by hypervisor software.

Types

- 1) Full virtualization.
- 2) Para Virtualization

Paging

- * Physical address space of a process can be noncontiguous: process is allocated physical memory.

- * Avoids external fragmentation by dividing physical memory into fixed-size ~~memory chunks~~ blocks called frames.

* Paging splits up address space into fixed-sized unit called a page.

* Physical memory is also split into some number of pages called page frame.

* Page Table (Virtual $\rightarrow$ Physical address).

Page 1
Page 2
Page 3
Page 4
Page 5

(Page Table)

8.5
Frame 1
Frame 2
Frame 3
Frame 4
Frame 5

Main memory.

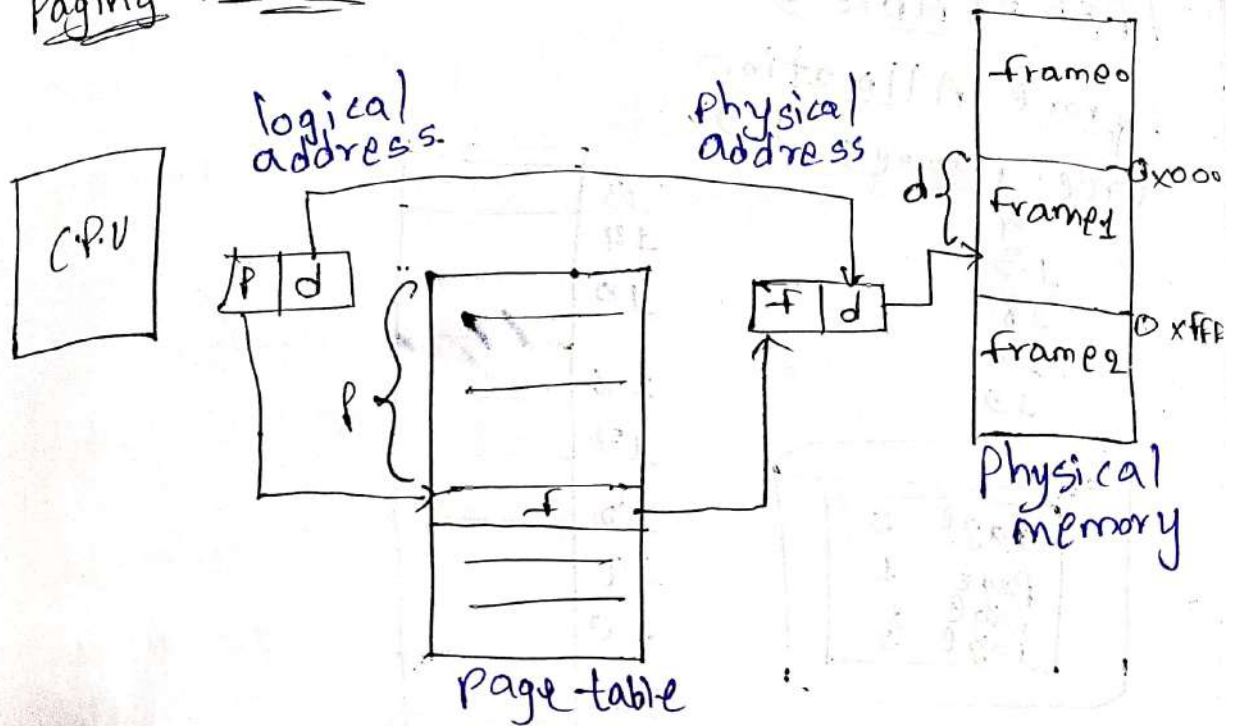
23/09/25

Page number:- used as an index into a page table which contains base address of each page in physical memory.

Page table:- Contains base address of each page in physical memory.

Page offset:- Combined with base address to define the physical memory address.

Paging Hardware



Paging Model of Logical and Physical memory

Page 0
Page 1
Page 2
Page 3

logical memory

0	1
1	4
2	3
3	7

Page table

0	
1	Page 0
2	
3	Page 2
4	Page 1
5	
6	
7	Page 3

physical memory

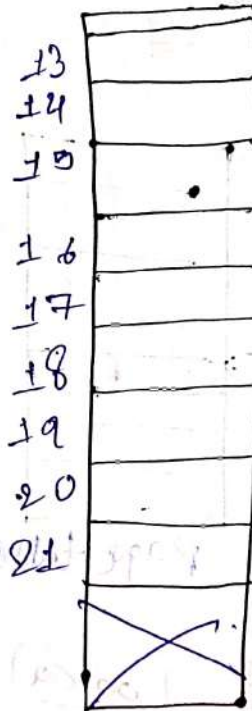
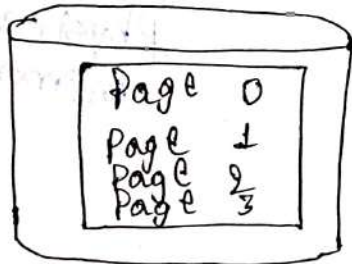
0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

0	
1	
2	
3	

FREE FRAMES

Before Allocation
free-frames list

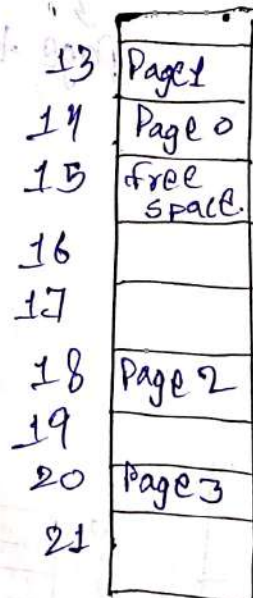
14
13
18
20
19



After allocation

free-frames list

15



PTBR (Page Table base register)

* points to the page table.

PT LRL (Page Table Length register)

* indicates size of the page table.

* Translation look-aside buffer (TLBs) are used to solve two-memory access.

Advantages (Paging)

- Easy to use memory management algorithm.
- No need for external fragmentation.
- Swapping is easy between

SEGMENTATION

- * A process is divided into segments.
- * The chunks that a program is divided into which are not necessarily all the same sizes are called segments.
- * User's view is mapped to physical memory.

Types of segmentation

→ Segment Table:-

Maps 2D-logical address into 1D Physical address.

* Each table ~~address~~ ^{entry} has:

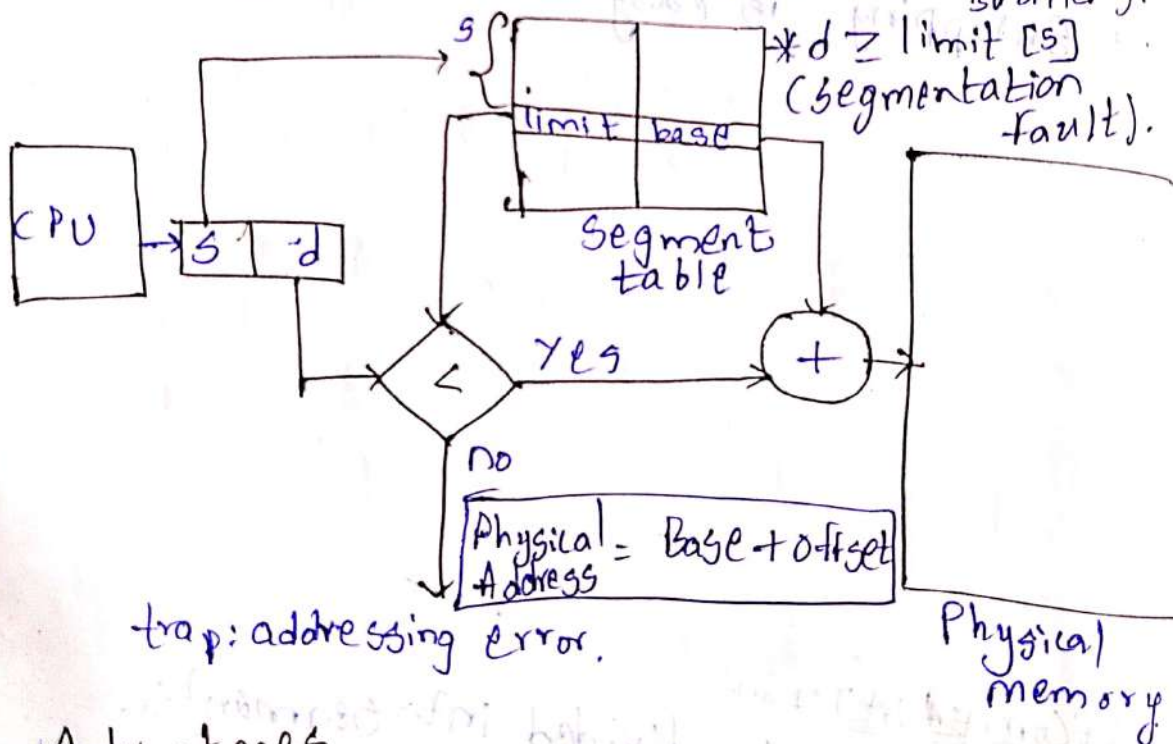
→ Base address:- Containing the starting physical address where the segments reside in memory.

→ Limit:- It specifies the length of the segment.

Segmentation Hardware

* $d < \text{limit}[s]$
offset (Then address is valid).

* $d \geq \text{limit}[s]$
(segmentation fault).



Advantages

* No internal fragmentation.

* Segment table consumes less space in comparison to page table in paging.

Disadvantages

→ The free memory space is broken into little pieces, causing external fragmentation.

Paging

- 1) Main memory is partitioned into frames.
- 2) Logical address space is divided into pages by the compiler or memory management unit (MMU).
- 3) Suffers from internal fragmentation or page breaks.
- 4) O.S maintains a free frame list; no need to create a free-frame.
- 5) The O.S maintains a page map table for mapping b/w frames & pages.
- 6) Does not support the user view of memory.
- 7) Processor uses the page number & displacement to calculate absolute address (p.d)

Segmentation

- 1) partitioned into segments.
- 2) Logical address space is divided into segments.
- 3) Suffers from external fragmentation.
- 4) O.S maintains the particulars of available memory.
- 5) The O.S maintains a segment map table for mapping purpose.
- 6) Supports user view of memory.
- 7) Processor uses the segment number and displacement to calculate the absolute address.

PAGE REPLACEMENT

14/10/23

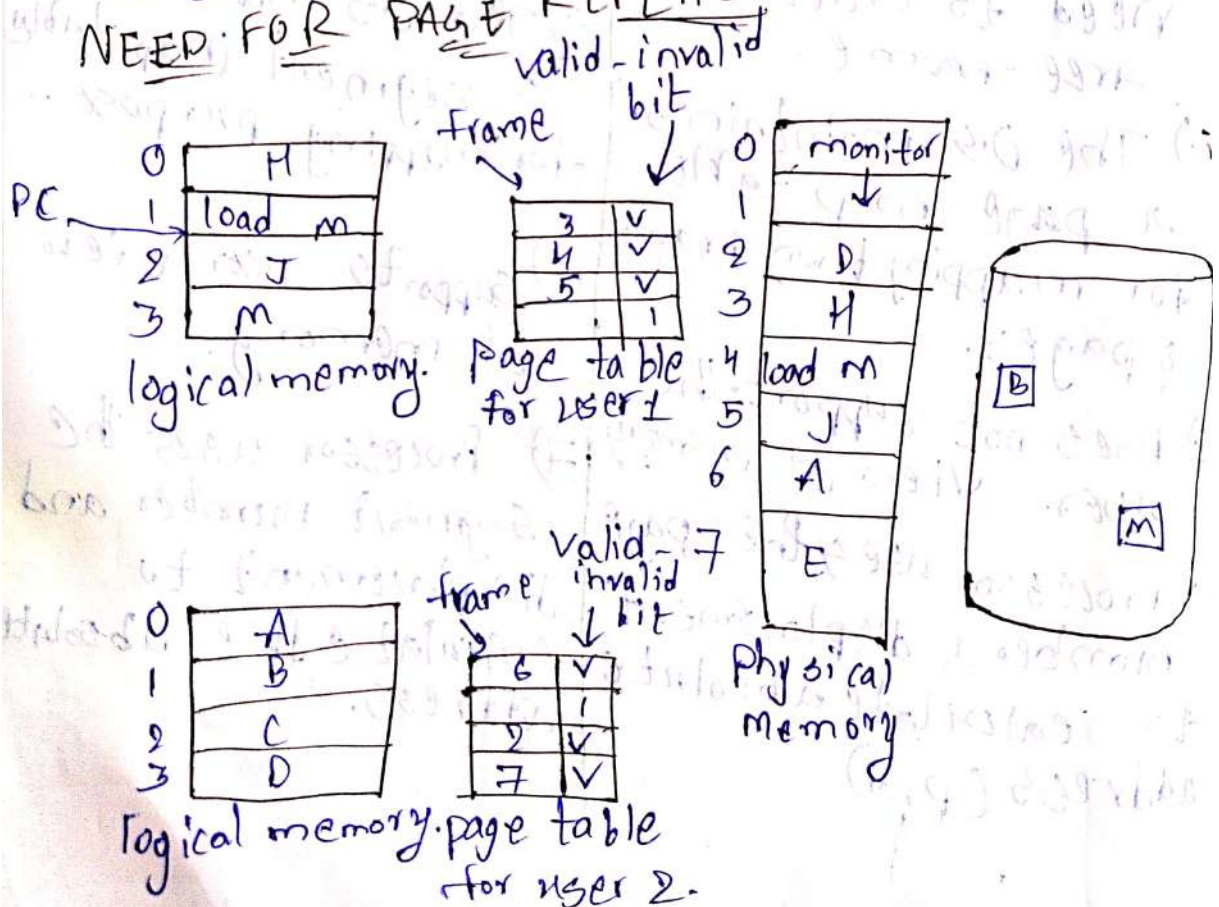
* In OS a page replacement algorithm is needed to decide which page needs to be replaced when a new page comes in.

* Used to manage memory efficiently when the physical memory is full.

* Prevents over-allocation of memory by modifying page fault service routine to include page replacement.

* Only modified pages are written to disk.

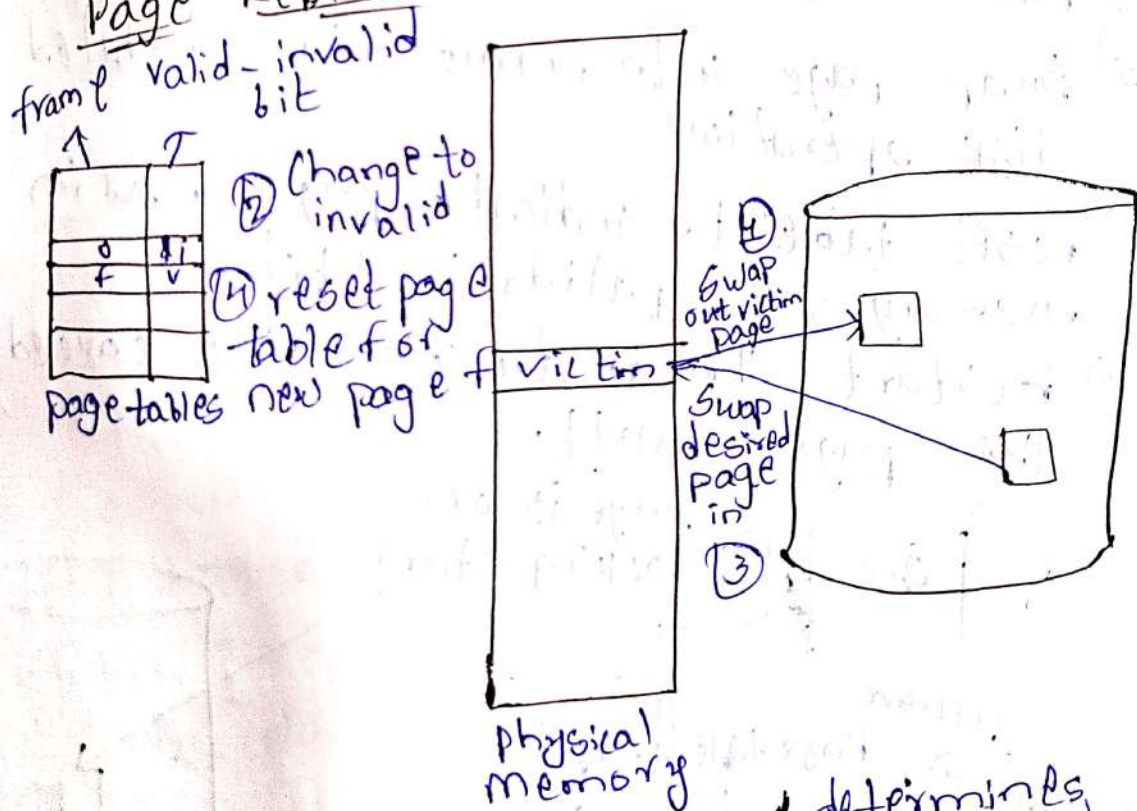
NEED FOR PAGE REPLACEMENT



Basic Page Replacement

- 1) Find the location of the desired page on disk.
- 2) Find a free frame,
→ If there is a free frame, use it
→ If there is no free frame, use a page replacement algorithm to select a victim frame.
→ Write victim frame to disk if dirty.
- 3) Bring the desired page into the free frame; update the page & frame tables.
- 4) Continue the process by restarting the instruction that caused the trap.

Page Replacement



Frame-allocation algorithm determines:

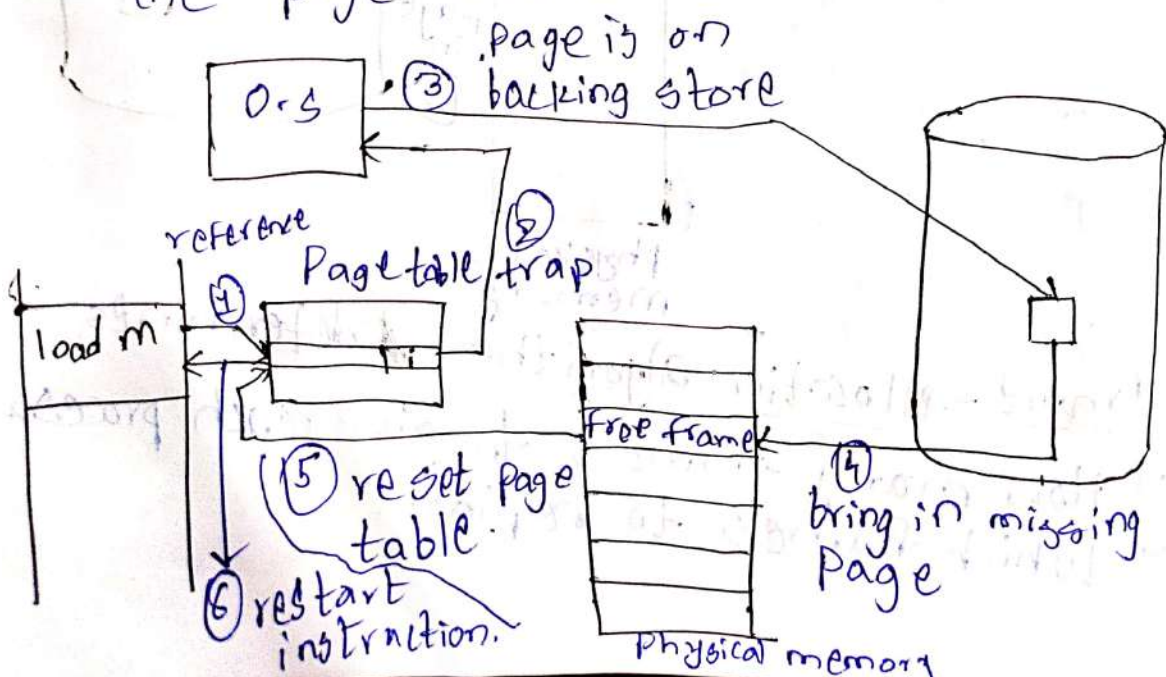
- How many frames to give each process
- Which frames to replace

Page replacement algorithm

- Want lowest page-fault rate on 1st & first access & re-access.
- String is just page numbers, not full addresses.
- Repeated access to the same page does not cause a page fault.

Page Fault

- 1) O.S looks at another table to decide:
 - Invalid reference $\Rightarrow$ abort
 - Just not in memory.
- 2) Find free frame
- 3) Swap page into frame via scheduled disk operation.
- 4) Reset tables to indicate page now in memory & set validation bit = V.
- 5) Restart the instruction that caused the page fault.

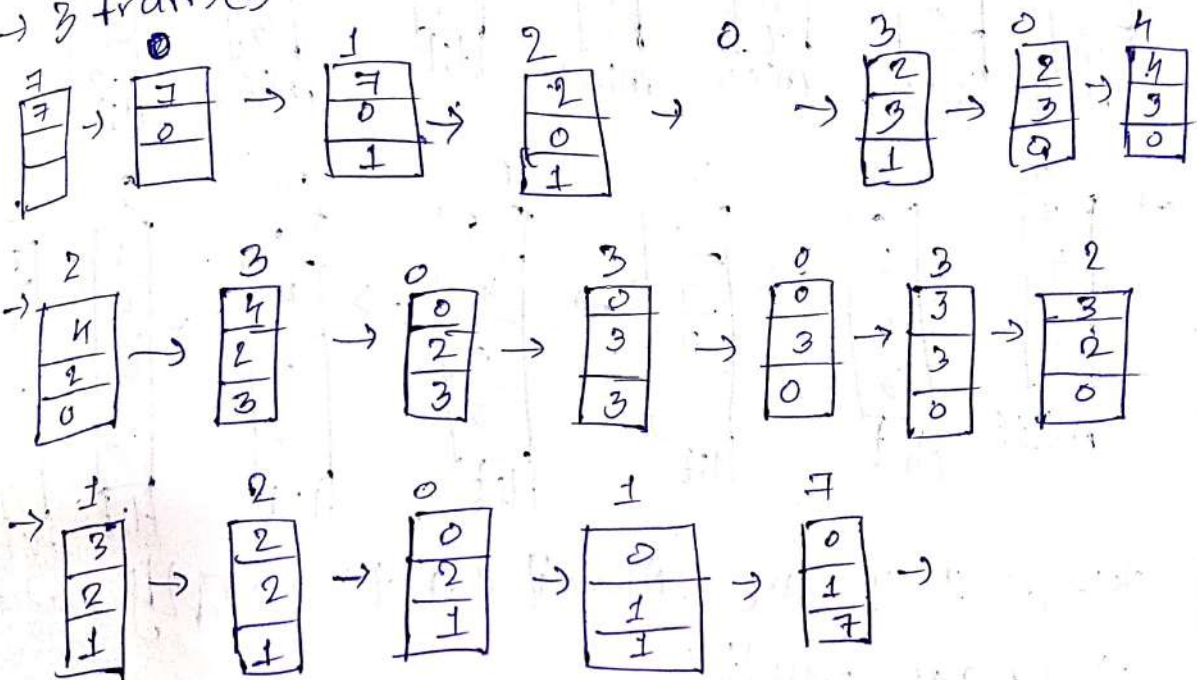


FIFO ALGORITHM

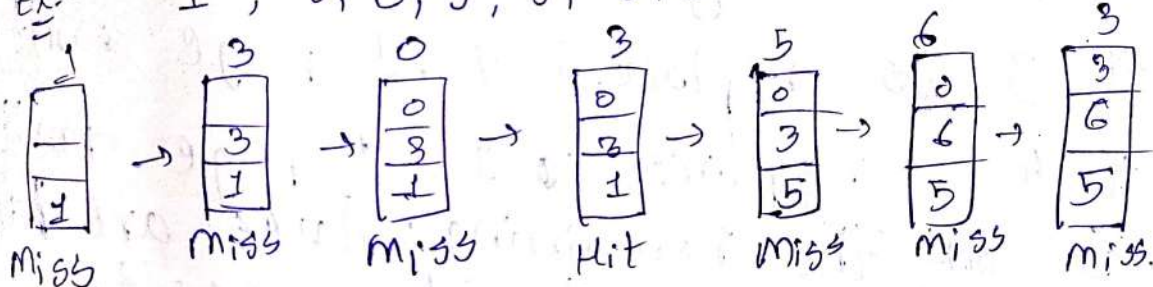
Ex

→ Reference String: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 3, 2, 1, 2, 0, 1, 7, 0, 1

→ 3 frames



Ex: 1, 3, 0, 3, 5, 6, 3



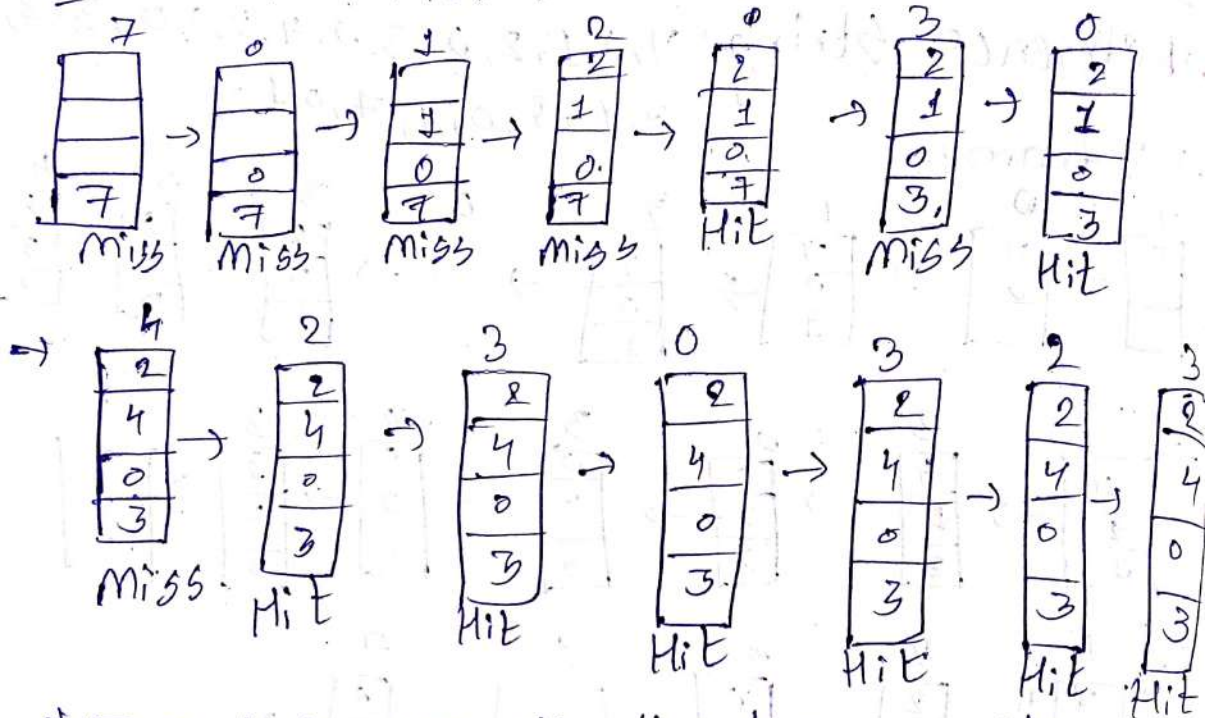
* 1, 3, 0 are allocated to the empty slots → 3 Page Faults.

* When 3 comes, it is already in the memory so 0 Page faults.

* Then 5 replaces 1, 6 replaces 3 & 3 replaces 0 each have 1 page fault.

Optimal Page Replacement Algorithm

Ex:- 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3



* 7, 0, 1, 2 are allocated to empty slots
 → 4 page faults.

* 0 is already present.

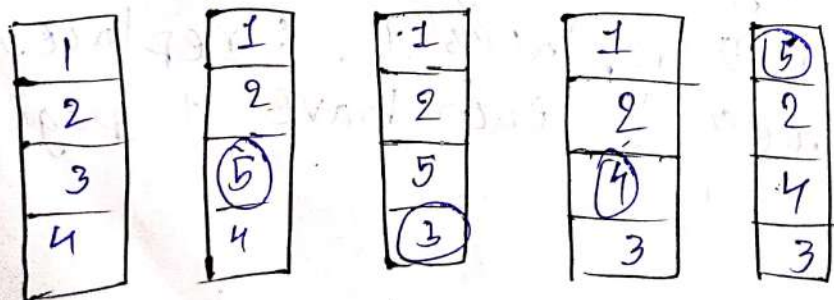
* 3 replaces 7, 1 page fault.

* 4 replaces 0, 1 page fault.

∴ all the remaining values are hit
 i.e; which are already present.

Least Recently Used (LRU)

Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

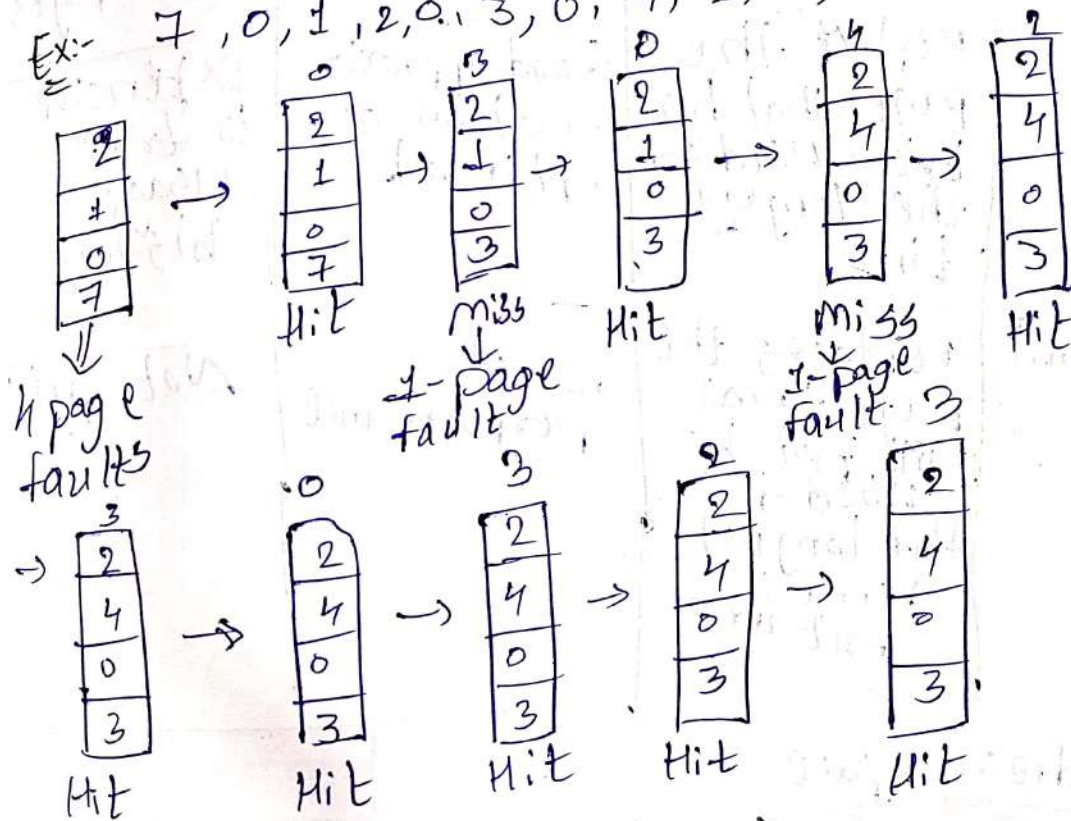


* Here 4, 1, 2 are recently used before 5, so replace 5 ~~with~~ in the place of 3.

* Next 5, 1, 2 are recently used before 3, so replace 3 in the place of 5.

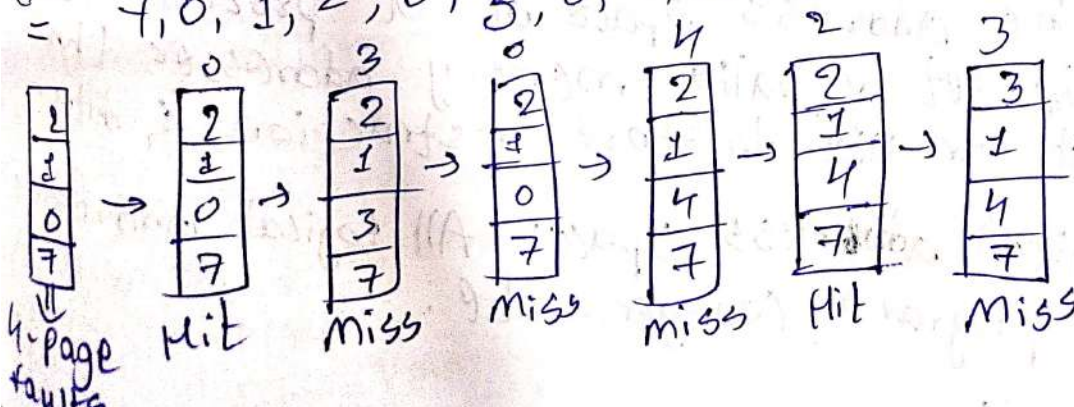
* At last 2, 3, 4 are recently used before 5, so replace 5 in the place of 1.

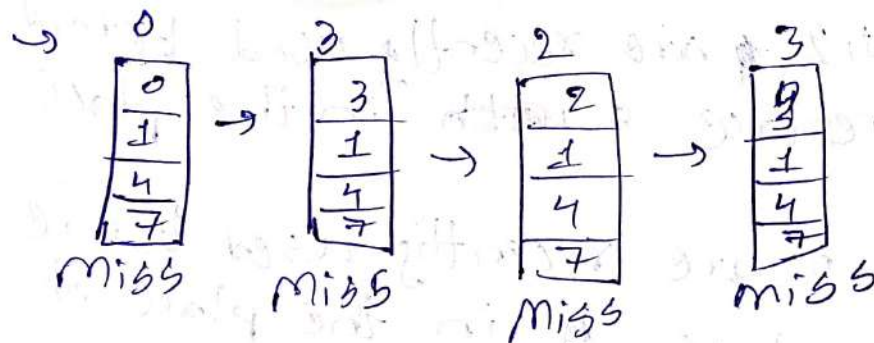
Ex:- 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3



Most Recently Used (MRU)

Ex:- 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3





* 12 page faults.

Algorithm	Description	Pros	Cons
FIFO	Replaces the oldest loaded page	Simple to implement	May replace frequently used pages
LRU	Replace the page that hasn't been used for the longest time.	Good approximation of optimal	Extensive to track usage history.
Optimal	Replaces the page that will not be used for the longest time in future	Best performance	Not practical

Address space

* The range of logical addresses that the O.S assigns to a process.

* The address space of a process is the set of valid memory addresses that it can use to store instructions & data.

Logical address space: All logical addresses a program can generate.

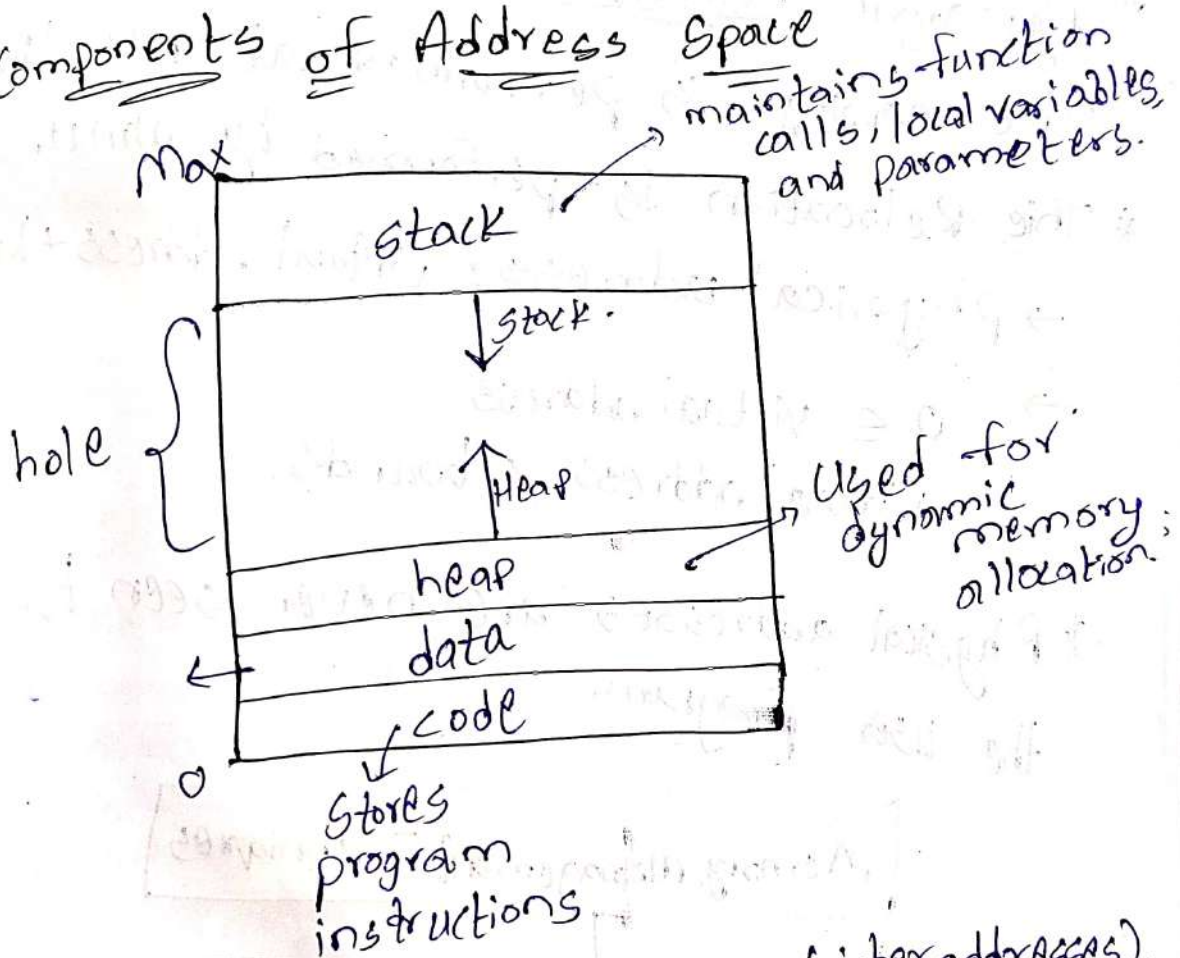
Physical address space: All physical addresses corresponding in memory.

Binding Schemes

Compile-time & Load-time: Logical = Physical

Execution-time: Logical $\neq$ Physical

Components of Address Space

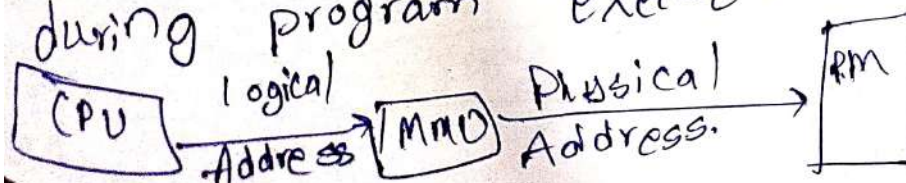


* Heap grows upward (towards higher addresses).

* Stack grows downward (from max address).

Memory Management Unit (MMU)

* The hardware that converts virtual addresses to physical addresses dynamically during program execution.



Relocation is way to map virtual addresses into physical addresses.

Types of relocation

1) Static Relocation:- This is performed before or during the loading of the program into memory, by relocating linker/loader

2) Dynamic Relocation:-

The mapping is performed at run-time

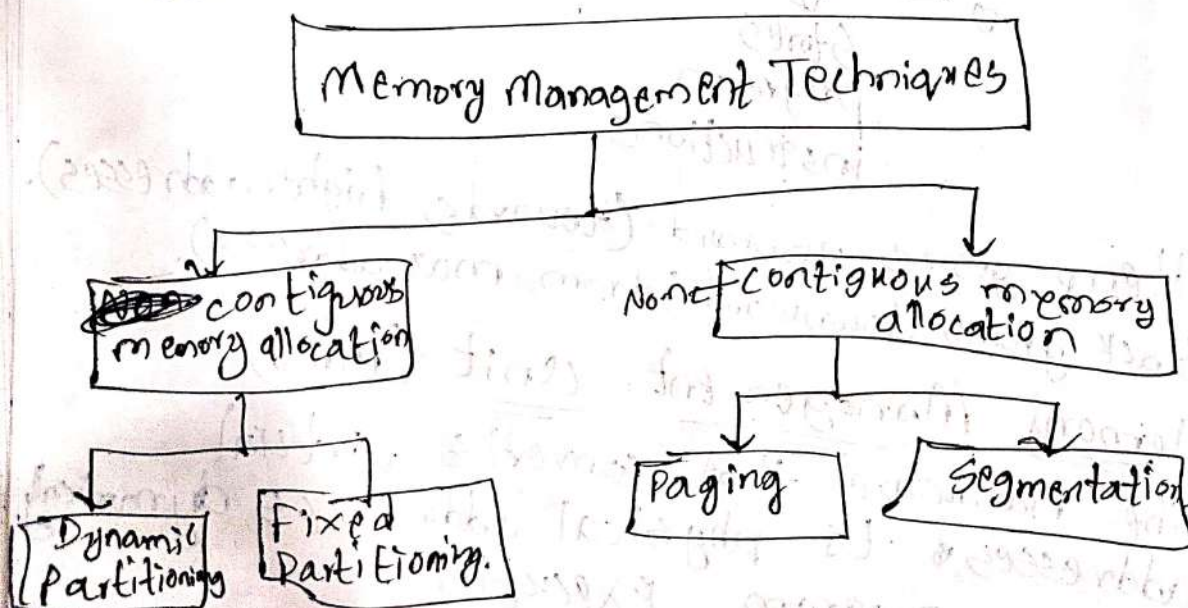
* This Relocation is performed by MMU.

→ physical address = virtual address + base

→ $0 \leq \text{virtual address}$

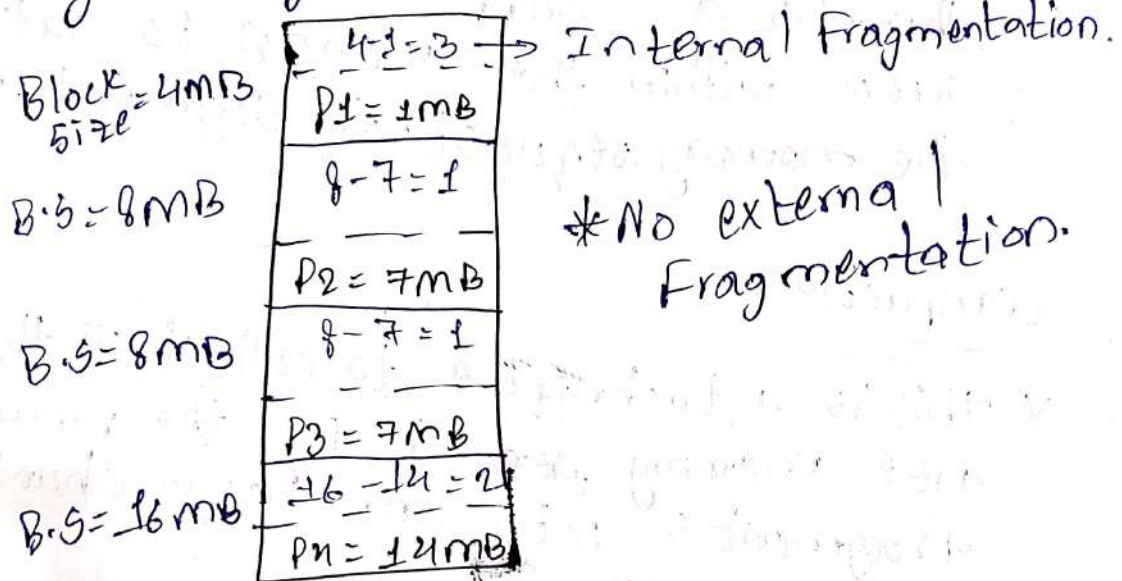
virtual address $<$ bounds.

* Physical addresses are never seen by the user program.



Fixed Partitioning

* In this, the number of partitions in RAM is fixed, but each partition's size may or may not be the same.

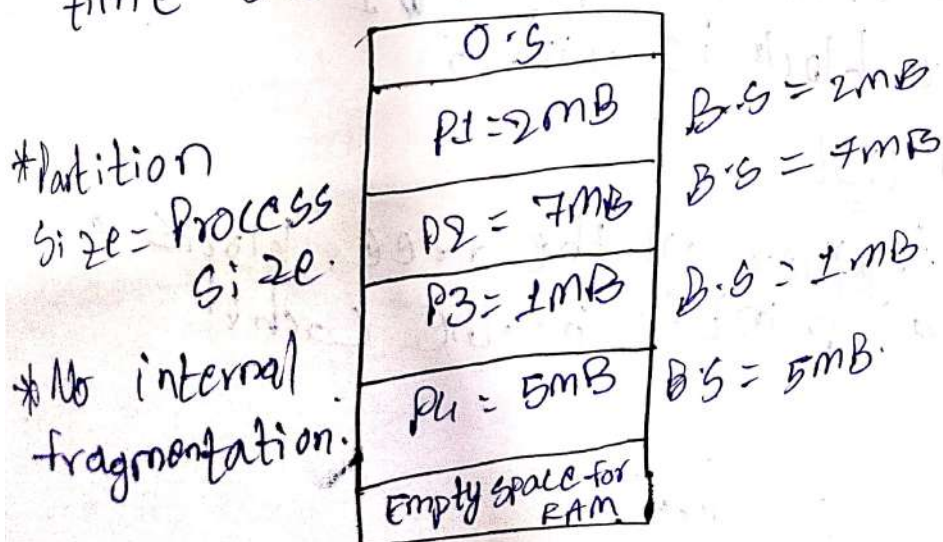


$$(4-1) + (8-7) + (8-7) + (16-14) = 3+1+1+2 = 7\text{MB.}$$

Dynamic Partitioning (Variable)

* Unlike fixed partitioning, partitions are not made before the execution or during system configuration.

* Partitions are made during the run-time according to the process's.



* Internal fragmentation happens when the memory is split into mounted-sized blocks. $\Rightarrow$ paging

* External fragmentation happens when there's a sufficient quantity of area within the memory to satisfy the memory request. $\Rightarrow$ segmentation.

Compaction

* This is a technique to collect all the free memory present in the form of fragments into one large chunk of free memory.

* This is done when relocation is dynamic & done at execution time.

Free Space Management

1) Bit Vector

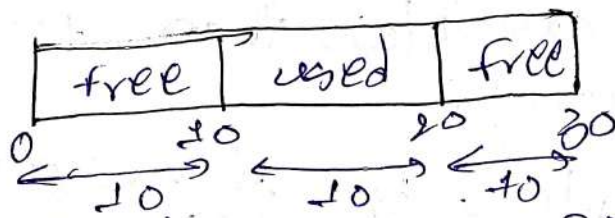
* The free space list is implemented as a bit map vector.

* If the block is empty = 1

If the block is full = 0.

2) Linked List

* Links together all the free blocks & keeping a pointer in the cache.



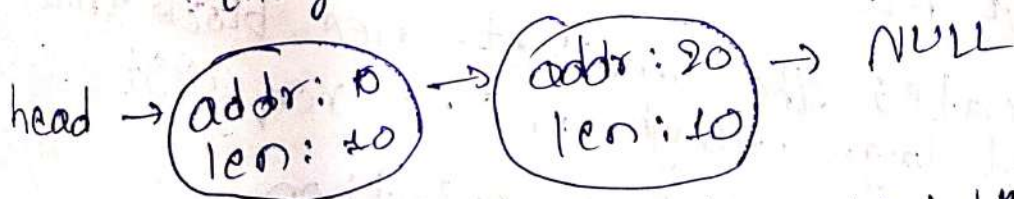
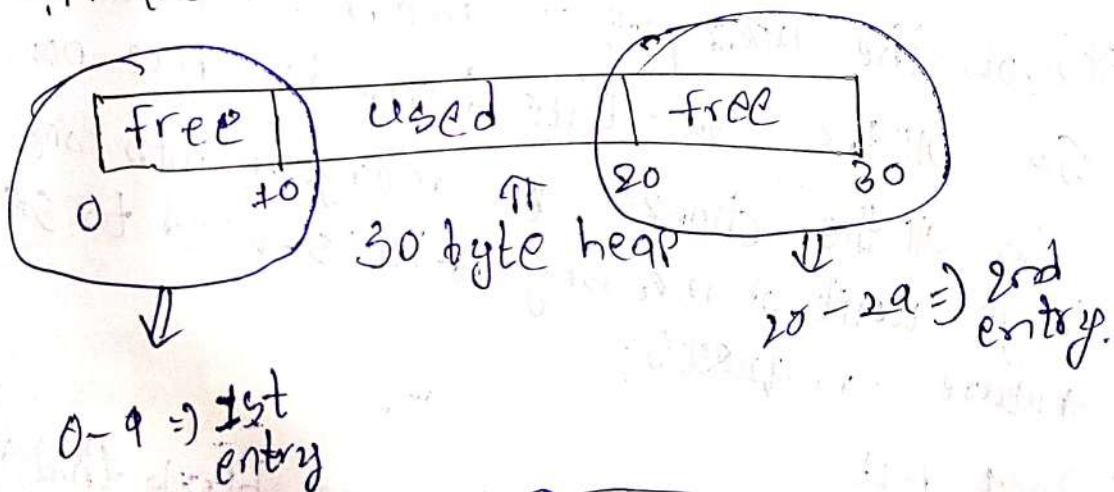
* Even though there is 20 bytes free space, the 15 byte request cannot be taken as the memory is divided into 10 bytes. This is external fragmentation.

Low level mechanisms for free space management.

- Splitting
- Coalescing.

Splitting:-

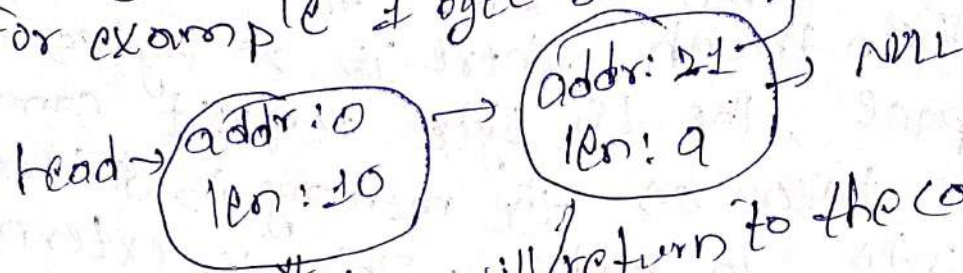
* A free list contains a set of elements that describe the free space still remaining in the heap.



* The requests greater than 10 bytes will fail, only the requests with exact size of 10 bytes will be satisfied.

* But in the case of less than 10 bytes, splitting occurs.

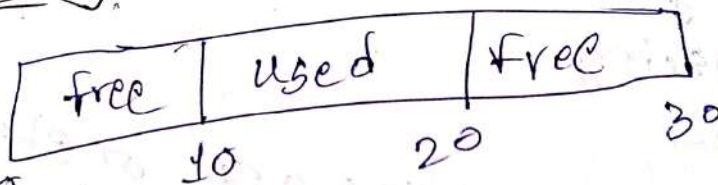
* For example 1 byte of memory, then



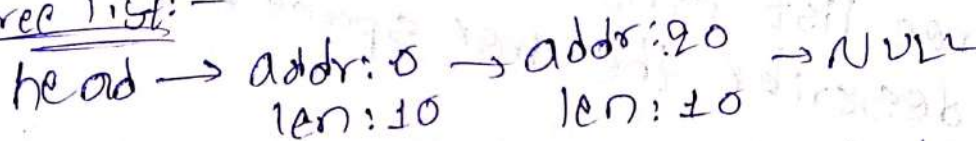
The first chunk - will return to the caller

The second chunk - will remain on the list

Coalescing



Free list -



* Now the used part is freed or deallocated

So all the 30-byte heap is free now.

So all the chunks are merged into one large unit of memory which is used to satisfy future requests.

Best Fit

* Allocates the smallest free block that is just large enough for the request.

* Less internal fragmentation.

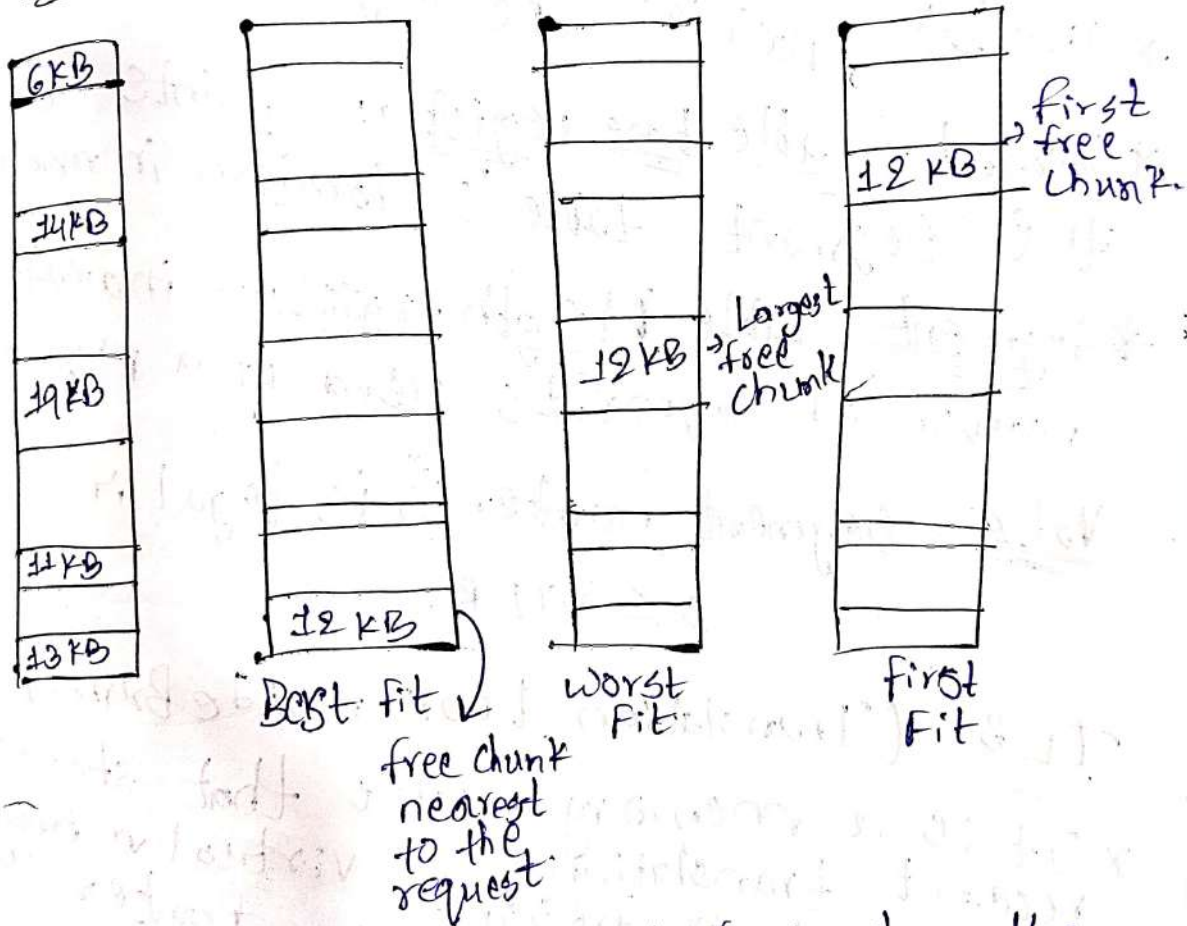
worst fit

- * Allocates the largest free block to the process.

first fit

- * Allocates the first free block it encounters that is large enough to satisfy the request.
- * This is faster.

Ex:-



Next fit:-

Starts the search from where the previous search ended, not from the beginning.

Segmentation

* A logical address consists of a two tuple.
 $\langle \text{segment-number, offset} \rangle$

* Segmentation table $\Rightarrow$ It maps two-dimensional logical address into one-dimensional physical address.

* base $\Rightarrow$ contains the starting physical address where the segments reside.

* limit $\Rightarrow$ specifies the length of the segment.

* Segment-table base register: - points to the segment table's location in memory.

* Segment-table length register: - indicates number of segments used by a program.

Note: - segment number s is legal if
 $s < \text{STLR}$.

TLB (Translation Look Aside Buffer):

* It is a memory cache that stores recent translations of virtual memory to physical addresses for faster retrieval.

* TLB consists two parts.

$\rightarrow$ Key/Tag.

$\rightarrow$ ~~Tag~~ Value.

Swapping

* This temporarily removes a process from memory & stores it on disk, freeing up physical memory for other processes.

Swap-Out - Removes a process from RAM and adds it to the hard disk.

Swap-In - Removes a program from a hard disk & putting it back into the main memory or RAM.

→ Standard swapping.

Thrashing

* A process is said to be thrashing if it is spending more time in paging than in execution.

* The high paging activity is called as thrashing.

* Device drivers encapsulate device details.

I/O Hardware

Port = Connection point for device.

Bus = daisy chain or shared direct access.

↓
multiple devices
linked together.

* PCI bus common in PCs & servers,
PCI Express (PCIe)

* Expansion bus connects relatively
slow devices.

* Serial-attached SCSI (SAS) common disk
interface.

* Controller (host adapter) ⇒ electronics that
operate port, bus, device.

* Fibre Channel (FC) ⇒ Used to connect
servers to storage devices in data
centers.

Techniques for device management

- 1) Dedicated
- 2) Shared
- 3) Virtual.

Dedicated Devices

- * These devices in O.S device management are dedicated to only one job at a time until that job releases them.

Ex:- printers, tape drivers, plotters.

Shared Devices

- * Devices such as disks, drums, and most other Direct Access Storage ~~Access~~ Devices (DASD) may be shared concurrently.

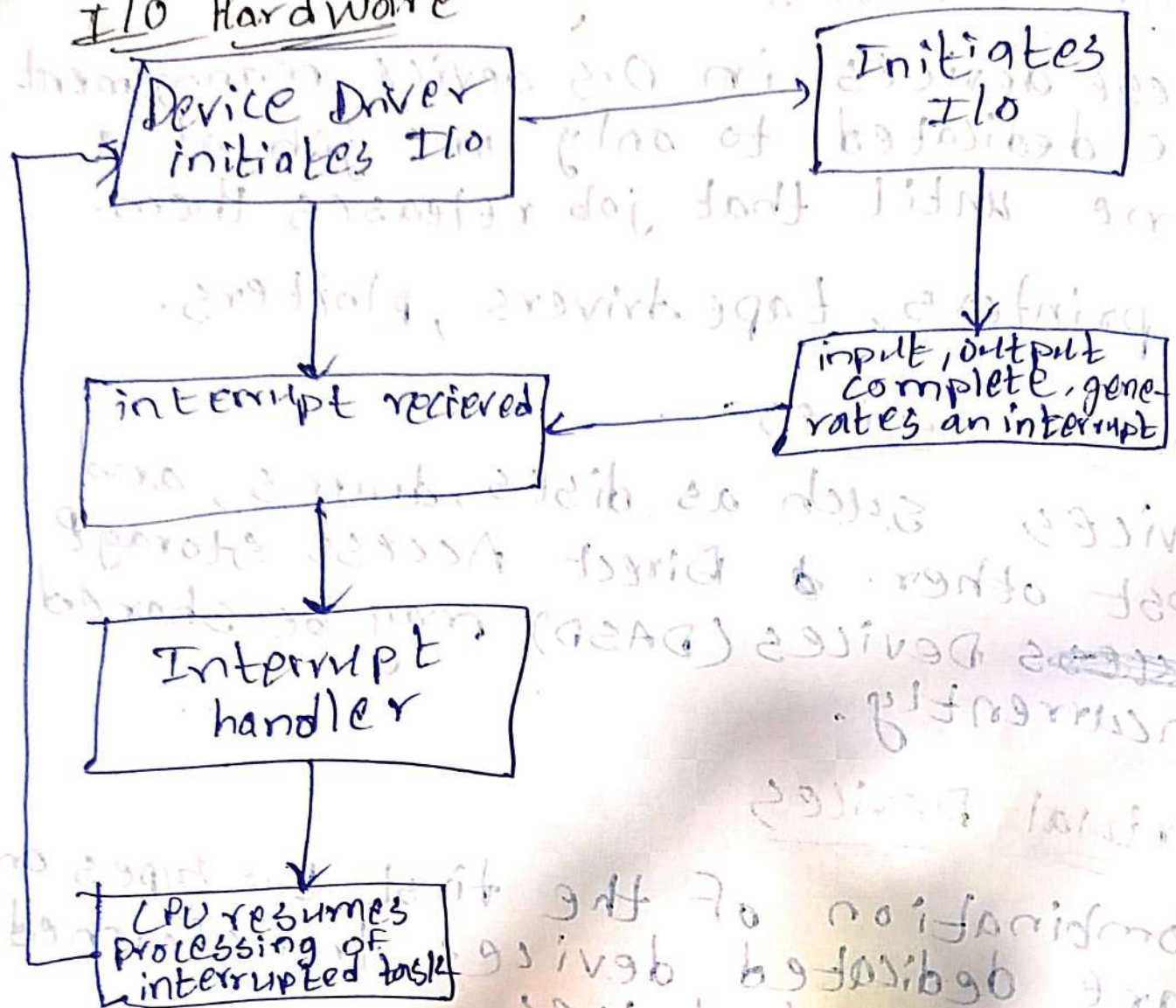
Virtual Devices

- * Combination of the first two types and are dedicated devices transformed into shared devices.

Spooling:- Re-routes all the print requests to a disk. A print job is not sent straight to the printer, instead, it goes to the disk (spool) until it is fully prepared with all the necessary sequences and formatting, and then it goes to the printer.

Buffering:- Overlap the I/O operation of one job with the execution of the same job.

I/O Hardware



Magnetic Disks

- * Sequential access storage device.
- * These provide a bulk of secondary storage in modern computer systems.
- * We store information by recording it magnetically on the platters.

Optical Disks

- * Electronic storage medium that can be written to or read from a low powered laser beam.

Direct Memory Access

- * Allows I/O devices to transfer data directly to or from the main memory (RAM), without involving the CPU.

Latency

- * Time delay between a cause and its effect (or) the time it takes for data to travel from one point to another.

Hard Disk Drives

- * Bulk of secondary storage for modern computers is hard disk drives.
- * Drives rotate at 60 to 250 times per second.
- * Range from 30 GB to 3TB per drive.
- * Transfer Rate $\Rightarrow$ 6 GB/sec
- * Effective transfer rate $\Rightarrow$ 1 GB/sec.
- * Latency based on spindle speed

$$\frac{1}{\frac{\text{RPM}}{60}} = \frac{60}{\text{RPM}}$$

$$\text{Average Latency} = \frac{1}{2} \text{ Latency.}$$

$$\text{Access Latency} = \text{Average seek time} + \text{Average Latency.}$$

$$\rightarrow \text{For fastest disk} = 3 \text{ ms} + 2 \text{ ms} = 5 \text{ ms}$$

$$\rightarrow \text{For slower disk} = 9 \text{ ms} + 5.56 \text{ ms} = 14.56 \text{ ms.}$$

Platter

- * Usually made of some hard material (aluminium), then coated with a thin magnetic layer.

Spindle

- * The platters are all bound together around the spindle.
- * RPM = 7,200 RPM to 15,000 range.

* Surface is divided into rings = tracks.

* Stack of tracks = cylinder.

Seek Time

- * It is the time it takes for the read/write head of a hard disk to move and position itself over the correct track.

$$\text{Time} = \text{Seek} + \text{rotation} + \text{transfer time}$$

HDD scheduling

* For using hardware efficiently - for the disk drives, this means having a fast access time and disk bandwidth.

* Disk bandwidth is the total number of bytes transferred, divided by the total time taken by the first

* This is done by O.S to schedule I/O requests arriving for the disk.

* Also known as I/O scheduling.

* Need to manage multiple I/O requests.

* Two or more request may be far from each other so can result in greater disk arm movement.

* Hard drives have to be accessed in an efficient manner.

Rotational Latency:- It is the time taken by the desired sector of disk to rotate into a position so that it can access the read/write heads.

Transfer Time:- Time taken to transfer the data.

Disk response Time:- Average of time spent by a request waiting to perform its I/O operation.

Ex:- Disk Tracks = 0 - 199

Request Queue = 82, 170, 243, 140, 24, 16, 190

Current Head Position = 55

seek time = $(82 - 55) + (170 - 82) + (140 - 43) + (240 - 43) + (140 - 24) + (24 - 16) + (190 - 16) = 637$ tracks.

SSTF (Shortest Seek Time First)

- * Requests having shortest seek time are executed first.

Ex:- 82, 170, 43, 140, 24, 16, 190.

Current position of Read/Write = 50

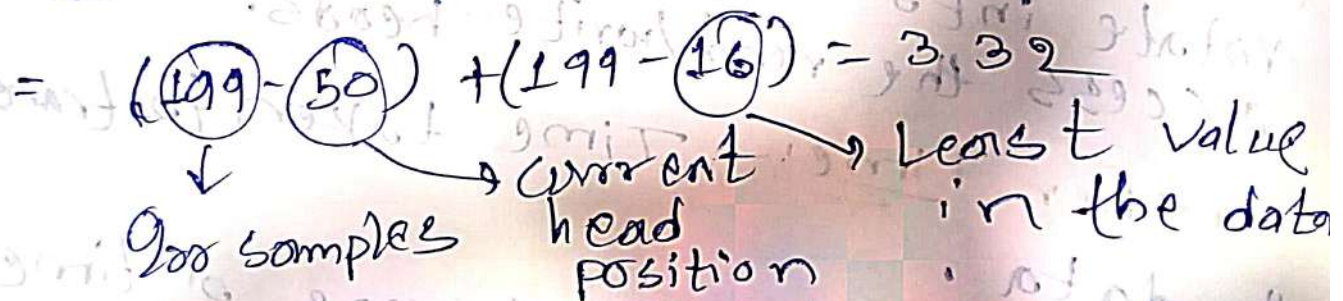
$$\begin{aligned} \text{Ex:- } & (50 - 43) + (43 - 24) + (24 - 16) + (82 - 50) \\ & + (140 - 82) + (170 - 140) + (190 - 170) \\ & = 208 \end{aligned}$$

SCAN

- * Also known as elevator algorithm.

Ex:- 82, 170, 43, 140, 24, 16, 190.

$$= (199 - 50) + (199 - 16) = 332$$



- * Higher throughput.

C-Scan

82, 170, 43, 140, 24, 16, 190

- * First from head position = 50

Moves to higher values = 50 $\rightarrow$ 199

50, (199 - 50)

- * Then moves to highest value to lowest value 50, (199 - 0).

* then 0 to below nearest value to 50. 50, (43-0).

$$= (199-50) + (199-0) + (43-0) = 391$$

10014

82, 170, 43, 140, 24, 16, 190.

$$= (190-50) + (190-16) = 314$$

↓
Highest
value
in the
data

↓
Head
position

↓
Lowest
value
in the
dataset

* Similar to SCAN algorithm.

~~SCAN~~ LOOK

* Similar to SCAN algorithm.

82, 170, 43, 140, 24, 16, 190.

$$(190-50) + (190-16) + (43-16) = 311$$

Storage Attachment

→ Computer's access storage in three ways.

- host - attached
- network - attached
- cloud

RAID (Redundant Array of Independent Disks)

- * This is a setup consisting of multiple disks for data storage.
- * Linked together to prevent data loss and to speed up performance.

Disk striping

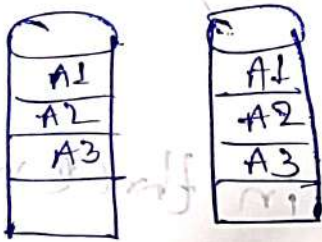
- * It uses a group of disks as one storage unit.
- * RAID is arranged into six different levels.

RAID 0: STRIPING

→ All the data is divided into blocks.

RAID 1: MIRRORED DISKS.

→ All data is stored twice.



we use hamming code
Parity method.

RAID 2:

the error of the data is checked at every bit level.

RAID 3:

At this level we store parity information in a dedicated parity drive.

RAID 4:

consists of block level striping with a dedicated parity disk.

RAID 5: (STRIPED DISKS WITH SINGLE PARITY).

* Requires at least three drives.

RAID 6: (STRIPING WITH DOUBLE PARITY)

* Data is written on two drives instead of one.

* Requires minimum four drives.

File

* A file is a collection of related data stored on a storage device.

* It is the basic unit of storage.

file Attributes

- Name
- Identifier
- Type
- Location
- Size
- Protection

File operations

- Create
- Write
- Read
- Delete
- open
- close

File types

- Regular files
- Directory files
- Special files.

File Access methods

- sequential access = Read/write in order
- Direct Access = Jump to any part directly.
- Indexed Access = Index is maintained to locate data blocks.

File locking

- Shared lock ⇒ Reader lock.
- Exclusive lock ⇒ Writer lock.

Disk structure

- * This describes the physical organization of a hard disk drive magnetic into concentric tracks, which are further divided into sectors, where data is stored.

Directory

- * A directory is a file system structure used to organize files and other directories hierarchically.
- * Organization
- * Hierarchy
- * Path Resolution
- * Access Control

Operations

- Search for a file
- Create a file
- Delete a file.
- Rename a file.

Single-Level Directory

- * All files are stored in a single, flat directory without any subdirectories or hierarchical folders.

Two-level directory

- * Separate directory for each user.

Ayclic Graph

- * A graph with no cycles.
- * You cannot return to the same node.

Ex:- $A \rightarrow B \rightarrow C$

Cyclic Graph

- * Graph contains cycles.

Ex:- $A \rightarrow B \rightarrow C \rightarrow A$

General Graph

- * Can contain both cycles and ayclic parts.

- * Does not have any kind of restrictions.

File Structure

- Logical storage unit.
- Collection of related information.

Layered File System

The different functions are organized into layers.

Structure:-

- Logical File system = manages metadata information
- File organization module.
- Basic File system.
- I/O control
- Physical Devices.

File-system operations

- Boot-control block → contains info needed by system to boot OS from that volume.
- Volume control block → contains volume details.

Directory Implementation

- * Linear list of file names with pointer to the data blocks.
- * Hash Table → Linear list with hash data structure.

Directory Allocation Methods

- 1) Contiguous Allocation.
- 2) Linked Allocation
- 3) File Allocation Table (FAT).

Distributed system

- * It is a collection of loosely coupled nodes interconnected by a communications network.

* Nodes variously called processors, computers, machines, hosts.

Ex - When we search "Weather today" on goog
→ Our request gets to a google server.
→ That server forwards it to thousands of distributed systems or machines.

* Each machine stores a small part of the web, they work in parallel to,

→ search indexes,

→ rank results

→ filter spam

* The combined results are sent to back to our browser.

⇒ Distributed systems exist in client-server, peer-to-peer, or hybrid configuration.

* In client-server configuration, server has a resource that a client would like to use.

* In peer-to-peer configuration, each node shares equal responsibilities & can act as both clients & servers.

* Communication over a network ^{occurs} through message passing.

Advantages

→ Resource sharing

→ Computation speedup.

→ Load balancing

→ Reliability.

Network structure

→ LAN (Workstations, laptops, mobile devices).
(printers, storage arrays).

→ WAN (Telephone lines, leased lines, optical cable, microwave links, radiowaves).

Communication Protocol

* This has three multiple layers

→ Layer 1 :- Physical Layer ⇒ handles mechanical & electrical details of the physical transmission of bit stream.

→ Layer 2 :- Data Link Layer ⇒ handles the frames, or fixed length parts of packets.

→ Layer 3 :- Network Layer ⇒ provides connections & routes packets in the communication network.

→ Layer 4 :- Transport Layer ⇒ responsible for low-level network access & for message transfer between devices to exchange routing information so that data packets can be forwarded.

→ Layer 5 :- Session Layer ⇒ implement process-to-process communication protocols.

→ Layer 6 :- Presentation Layer ⇒ resolves the differences in formats among the various sites in the network.

→ Layer 7 :- Application Layer ⇒ interacts directly with users.

User Datagram Protocol

- * This is a connectionless, fast, and lightweight communication protocol used on the Internet.
- * This a communication protocol that sends data packets quickly without checking whether they reach the destination or not.
- * UDP packets are also called datagrams.

Transmission Control Protocol

- * It is a protocol that ensures data is delivered correctly, in order without errors.
- * If something goes wrong, TCP fixes it automatically.
- * Connections are initiated with series of control packets called a three way handshake.

Network Operating Systems

- * This is designed to manage, operate, & support network resources, allowing computers to communicate.

Distributed Operating Systems

- * Manages a group of independent computers and makes them appear to the user as one powerful computer by coordinating their resources & tasks.

Data Migration

- * Transfer data by transferring entire file.

Computation Migration

A running process is moved from one machine to another in the network so that the work can be performed more efficiently.

Process Migration

- * Executes an entire process, or parts of it, at different sites.

Issues of Distributed Systems

- * Robustness
- * Transparency
- * Scalability

Distributed File System

- * A file system whose clients, servers, and storage devices are dispersed among the machines of a distributed system.

Data Integrity
* The overall precision, completeness, & continuity of data is known as data integrity.

Disk Failure modes

* Disk failures can be in two ways,

1) Complete disk failure = The whole disk stops working.

2) Latent - sector errors (LSEs) = only small portions of the disk become unreadable.

* High frequency of LSEs $\Rightarrow$ Disk is degrading, likely to fail soon.

* Low frequency of LSEs $\Rightarrow$ Disk is healthy.

* Higher the number of LSEs, the greater the chance of,

$\rightarrow$ Data loss

$\rightarrow$ RAID rebuild failures

$\rightarrow$ Disk crashing completely.

$\rightarrow$ Slow read/write operations.

$\rightarrow$ Silent corruption.

Block Corruption

* This is a situation where a block of data on storage becomes damaged, unreadable, or contains incorrect data.

Reasons

1) Disk hardware failure

2) Power failure during write.

- Bad memory
- Software bugs.
- Overheating or aging.
- Firmware or driver issues.

* Latent sector errors are easily detected & handled using redundancy mechanisms. such as RAID or RAID-4 & RAID-5 system based on parity.

Checksum function

* It is an algorithm that takes input data and produces a numerical value that represents that data.

* If even one bit changes, the checksum also changes $\Rightarrow$ error detected.

$\Rightarrow$ XOR

* A checksum where all bytes/words are XORed together to produce a final value.

* Detects simple errors like flipped bit

* Fast, simple - but weaker error detection

$\Rightarrow$ Addition

* Adds all the data bytes/words & taking the lower bits of the result.

* Stronger than XOR, but still not very robust

Ex:- $\text{checksum} = (10 + 20 + 30 + 40) \bmod 25$

$$100 \bmod 256 = 100$$

* It fits into 1 byte (8 bits)

$$(1 \text{ byte} = 8 \text{ bits} = 2^8)$$

Fletcher Checksum

* This uses two running sums, making it much better at catching errors.

S_1 = sum of all bytes (8-bit)

S_2 = sum of S_1 values (8-bit)

$$\text{checksum} = (S_2 \ll 8) | S_1$$

left shift S_2 by 8 bits

both

(1) $\Rightarrow$ bitwise OR operator combines the byte to form a final 16-bit checksum.

Cyclic Redundancy Check

* Very strong error-detection code that treats data as a binary number & performs polynomial division using modulo-2 arithmetic.

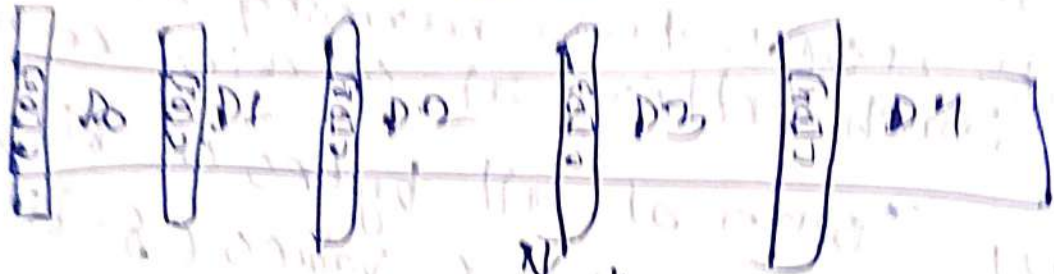
* Strongest checksum.

Ex - Used in Ethernet, ZIP files, disk drives, network packets.

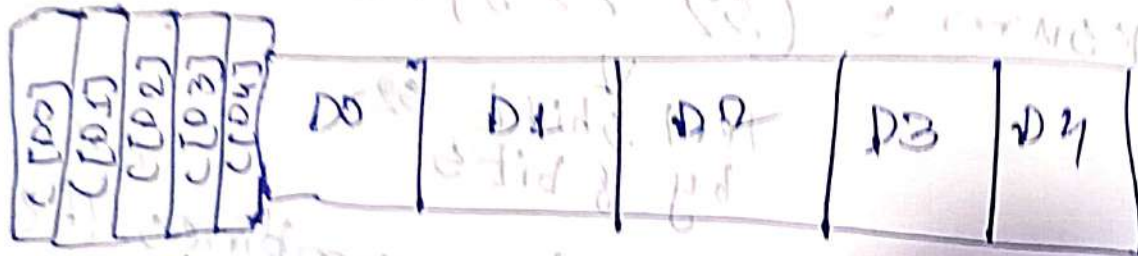
Checksum Layout



without
checksums



with
checksums.



checksums packed into 32-bit blocks.

Lost write

This happens when a write request is acknowledged as successful, but the actual data is not written to disk due to hardware, software, or caching issues.

Scrubbing

* It is the background process of reading all data on a storage system, verifying it using checksums/parity, and correcting any errors automatically.